Transportation Research Part E 209 (2026) 104760

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

LLM-based automatic heuristic design for vehicle-drone collaborative routing problems

Haiyang Shi, Lu Zhen *

School of Management, Shanghai University, Shang Da Road 99, Shanghai 200444, China

ARTICLE INFO

Keywords:
Large language models
Automatic heuristic design
Self-debugging mechanism
Vehicle-drone collaborative routing

ABSTRACT

Vehicle-drone collaborative routing problems (VDCRPs) have attracted increasing attention due to their potential to enhance logistics efficiency. However, designing effective algorithms remains challenging due to their structural complexity and the diversity of variants. This study presents a novel automatic heuristic design framework, termed LLM-VD, which integrates the text generation capabilities of LLMs with an evolutionary computation framework. Through structured prompting strategies, the framework efficiently generates, evolves, and evaluates VDCRP heuristics. A key innovation of LLM-VD is the integration of two complementary self-debugging mechanisms. The first incorporates structured failure feedback into subsequent prompts to prevent recurring errors, while the second employs repair prompts that pair failed heuristics with their feedback to guide reflective algorithmic correction. Experiments on three VDCRP variants show that LLM-VD can generate high-quality heuristics that outperform the baseline heuristic and generalize well across various LLMs. The two self-debugging mechanisms significantly enhance the success rate, achieving nearly a fourfold improvement for the most complex problem variant. These findings underscore the potential of evolutionary computation and self-debugging mechanisms in improving the robustness and adaptability of automatic heuristic design frameworks. LLM-VD provides a promising approach for VDCRPs and can be extended to other complex combinatorial optimization problems.

1. Introduction

With the rapid advancement of low-altitude flight technologies, unmanned aerial vehicles (UAVs), commonly referred to as drones, have been increasingly applied across logistics, military operations, agricultural production, and public services, demonstrating unprecedented flexibility and responsiveness. In particular, in logistics (Moshref-Javadi and Winkenbach, 2021; Wu et al., 2023), vehicle-drone collaborative delivery models leverage the endurance and load capacity of ground vehicles alongside the agility and speed of drones, enabling fast responses to customer demands without additional infrastructure. For instance, in densely populated urban areas, drones can reduce delays caused by traffic congestion (Paul et al., 2025; Zhen et al., 2025). In contrast, in rural regions or disaster zones, drones can overcome terrain and road limitations to deliver emergency supplies rapidly, even under damaged infrastructure or scarce resources (Lu et al., 2025; Yin et al., 2023). These advantages have motivated the increasing adoption of vehicle-drone collaboration in practice, establishing the research of VDCRPs as an emerging and promising research field. However, due to the highly dynamic application environments and the NP-hard nature of routing problems, customized modeling and efficient solution

* Corresponding author.
E-mail addresses: drshy@shu.edu.cn (H. Shi), lzhen@shu.edu.cn (L. Zhen).

https://doi.org/10.1016/j.tre.2026.104760
Received 30 September 2025; Received in revised form 23 January 2026; Accepted 10 February 2026
Available online 20 February 2026
1366-5545/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

approaches for VDCRPs remain a formidable challenge.

In real-world scenarios, the complexity of vehicle-drone coordination grows sharply as problem size expands and scenario-specific constraints increase. The exponential growth of the solution space renders commercial solvers inadequate. Consequently, current solution methods for VDCRPs largely rely on handcrafted designs, including exact algorithms, *meta*-heuristics, and learning-based methods. Exact methods, such as branch-price-and-cut (Schmidt et al., 2025; Yang et al., 2025), can provide optimal solutions but are limited to small-scale problems. Meta-heuristics, including variable neighborhood search (De Freitas and Penna, 2020), particle swarm optimization (Li et al., 2024), and large neighborhood search (Sacramento et al., 2019), offer practical effectiveness but lack problem-specific strategies and suffer from limited local exploitation. Learning-based methods, such as deep reinforcement learning (Bi et al., 2023), partially alleviate the reliance on manual design, yet often require large-scale pretraining and exhibit limited generalization. Overall, existing approaches fall short of providing a unified, expert-independent framework capable of automatically developing heuristics applicable across different VDCRP variants.

The emergence of large language models (LLMs) offers a new opportunity to address this challenge. With their strong contextual understanding and global reasoning capabilities, LLMs can transform complex optimization problems into interpretable natural language formats via structured prompting strategies, and automatically generate corresponding heuristics. LLMs thus hold promise as adaptive and flexible heuristic designers for complex and dynamic decision scenarios. Recent studies have mainly explored integrating LLMs with evolutionary computation (EC), where LLMs are used as heuristic generators and EC serves as a selection and refinement mechanism. Representative frameworks include the Evolution of Heuristics (EoH) proposed by Liu et al. (2023a) and FunSearch by Romera-Paredes et al. (2024). EoH and FunSearch pioneer the idea of evolving heuristics or program logic directly, rather than optimizing candidate solutions, thereby opening a new direction for automatic algorithm discovery. However, existing frameworks primarily rely on an elimination-based strategy, in which heuristics that fail evaluation are simply discarded in each iteration. This design implicitly assumes that the reliability of LLM-generated heuristics can be ensured through population-level filtering. In practice, due to the inherent limitations of LLMs in handling execution-level constraints and algorithmic correctness, a large proportion of generated heuristics are infeasible or erroneous, especially when applied to complex routing problems such as VDCRPs that involve multiple constraints and coordination mechanisms. Discarding these failed heuristics leads to low knowledge utilization and high heuristic design costs.

To improve the reliability and efficiency of LLM-based heuristic generation, we incorporate a self-debugging mechanism into the EoH framework. Instead of merely eliminating failed heuristics, the proposed approach extracts structured debugging information from evaluation failures and feeds this feedback back to the LLM. Guided by these signals, the LLM can repair invalid heuristics and avoid similar errors in subsequent generations. This self-debugging mechanism enables more effective use of previously generated heuristics, enhances constraint awareness, and improves the robustness of the generated heuristics, thereby making the proposed LLM-VD framework more suitable for complex and realistic VDCRPs. In conclusion, the main contributions of this study are summarized as follows:

1) This paper introduces LLM-VD, the first LLM-based automatic heuristic design framework specifically tailored for VDCRPs, which enables efficient heuristic generation without relying on traditional algorithm design or domain experts.
2) Structured prompting strategies and a rigorous evaluator have been designed for the heuristic generation and evolution process, guiding LLMs to generate heuristics adapted to different VDCRP variants.
3) Two self-debugging mechanisms are proposed to enhance heuristic generation success: one integrates failure feedback during evolution to prevent recurring errors, while the other employs code-level repair prompts to correct near-feasible heuristics.
4) Experimental results demonstrate that LLM-VD is capable of generating high-quality heuristics with strong generalization across different LLMs. Additionally, the self-debugging mechanisms significantly improve the success rate of heuristic generation, particularly for more complex VDCRP variants.

The remainder of this paper is organized as follows. Section 2 reviews the relevant work, and Section 3 provides the problem specification of VDCRPs. Sections 4 details our proposed methodology for VDCRPs automatic heuristic design. Section 5 presents the numerical experiments, and finally, Section 6 concludes the paper and discusses future research directions.

2. Related work

In this section, we primarily review relevant literature organized into two main research streams. The first stream deals with VDCRPs. And the second stream reviews LLMs-driven automatic heuristic design.

2.1. Vehicle-drone collaborative routing problems

A substantial body of research has been devoted to the field of VDCRPs. Researchers may refer to several systematic surveys, including Zhou et al. (2025), Dang et al. (2024), and Chung et al. (2020), for a comprehensive understanding of research progress in this field. Building on these studies, VDCRPs models are commonly classified into three categories according to the number of vehicles and drones involved: including the single-vehicle-single-drone (SVSD), the single-vehicle-multi-drone (SVMD), and the multi-vehicle-multi-drone (MVMD). These categories and their corresponding heuristic solution frameworks are summarized below.

The **SVSD** model is one of the earliest formulations, designed to combine the vehicle's carrying capacity and endurance with the drone's flexibility in last-mile operations. Here, all parcels are carried by the vehicle, which also serves as a mobile platform for

launching, retrieving, and recharging the drone. Representative formulations include the flying sidekick traveling salesman problem (FSTSP) by Murray and Chu (2015), where the vehicle supports drone deployment without requiring depot returns; the TSP with drone (TSP-D) by Agatz et al. (2018), which simplifies the setting by neglecting interaction times; and the VRP with drone (VRP-D) by Savuran et al. (2016), which set vehicles as the drone depot. These models are synchronous in nature, requiring coordinated operations between vehicle and drone, and are particularly suited to remote or sparsely distributed customer scenarios. The main challenge lies in assigning customers between the vehicle and drone under the drone's payload and range limits. While exact solvers may work for small cases, heuristic approaches dominate. Typical methods are TSP-based, reallocating feasible customers to drones from a baseline vehicle tour. Examples include route reallocation heuristics (Murray and Chu, 2015), route-first cluster-second strategies (Agatz et al., 2018), and hybrid methods such as local search and GRASP (Ha et al., 2018).

The **SVMD** model extends SVSD by allowing one vehicle to coordinate multiple drones, enabling parallel operations and reducing service time. First introduced as the parallel drone scheduling TSP (PDSTSP) by Murray and Chu (2015), SVMD generally assumes that vehicles and drones depart from the same depot, complete tasks, and return together. Synchronization constraints are often imposed for operational safety, such as launching drones only when the vehicle is stationary and retrieving them before the vehicle resumes travel (Kang et al., 2021; Moshref-Javadi et al., 2020; Salama et al., 2022). Conflict-free task execution is crucial, ensuring each customer is served by a single drone, often achieved through clustering geographically close customers into drone groups. Researchers have extended TSP-D and VRP-D to multi-drone variants such as TSP-mD (Kloster et al., 2023; Tiniç et al., 2023) and VRP-mD (Chen et al., 2021; Gu et al., 2022; Tamke and Buscher, 2021). Solution strategies frequently employ clustering-based decomposition: customers are split into central and peripheral nodes, with vehicles serving central nodes and drones handling peripheral ones. For instance, Chang et al. (2018) integrated K-means clustering with a TSP model to optimize cluster centers, expanding drone coverage while shortening vehicle tours.

The **MVMD** model is the most complex formulation, simultaneously considering multiple vehicles and drones (Gao et al., 2023a; Gao et al., 2023b). It requires decisions on drone and parcel allocation to vehicles, vehicle routes, drone sorties, and rendezvous points where vehicles and drones separate and rejoin. While operational constraints remain similar to simpler models—drones must be carried by vehicles and synchronized at rendezvous nodes—the presence of multiple vehicles greatly increases complexity. An additional assignment layer is needed to specify which drones and parcels are loaded onto which vehicles, yielding a highly coupled multi-decision optimization problem. Extensions of TSP-D and VRP-D to MVMD variants have also been explored by Montemanni et al. (2024) and Wang and Sheu (2019). Solution approaches typically model MVMD as VRP variants and rely on metaheuristics or decomposition strategies. A common procedure first generates feasible routes for multiple vehicles, then assigns drone sorties accordingly. For instance, Sacramento et al. (2019) applied Adaptive Large Neighborhood Search (ALNS) to a capacity-constrained MVMD variant, minimizing delivery time and costs. Wang et al. (2022) proposed an iterative local search framework that combines intra- and inter-route exchanges with re-optimization. Their method incrementally refines vehicle and drone routes by reassigning customers, improving both feasibility and efficiency. These studies highlight the complexity of MVMD problems and the importance of advanced heuristics to balance solution quality with computational effort.

Across all three categories, a common solution philosophy emerges: decomposing the problem into multi-stage and multi-level subproblems that are systematically integrated. The SVSD problem is typically solved in two steps: first generating a vehicle-only TSP tour, then assigning drone deliveries. The SVMD problem usually relies on clustering, with vehicles visiting cluster centers and drones serving surrounding nodes. MVMD problems extend this logic to three stages: generating multiple vehicle routes, allocating drone tasks along each route, and optimizing launch and rendezvous points for synchronized operations. This progression reflects the increasing structural complexity from SVSD to MVMD, as well as the growing reliance on sophisticated heuristics and decomposition to achieve practical solutions.

2.2. *LLMS-driven automatic heuristic design*

The emergence of LLMs has opened new avenues for automatic heuristic design. Traditional heuristics and metaheuristics often rely on handcrafted rules or substantial algorithm engineering, making them less adaptable to new problem variants. In contrast, LLMs can autonomously generate, refine, and adapt heuristic strategies through natural language interaction, code generation, and self-reflection, thereby reducing human effort and enhancing generality. Recent studies have explored the multifaceted roles of LLMs in optimization, ranging from direct solution generators to heuristic designers, demonstrating considerable potential in combinatorial optimization domains (Wu et al., 2025).

2.2.1. *Different roles of LLMs in algorithm design*

Liu et al. (2024) categorized the functions of LLMs in optimization into four roles: optimizer, predictor, feature extractor, and designer. As optimizers, LLMs can iteratively generate improved solutions by modeling decision variables and fitness values, showing competitiveness in both continuous and combinatorial optimization (Lange et al., 2024; Yang et al., 2023). As predictors, they can act as surrogate models to forecast performance indicators of candidate solutions (Jawahar et al., 2024). As feature extractors, LLMs generate embeddings and algorithmic features that support hyper-heuristic selection (Jiang et al., 2024). Most relevant to routing problems is their role as designers: LLMs have been employed to generate heuristic rules (Liu et al., 2024), formulate mathematical functions (Romera-Paredes et al., 2024), and even design mutation and selection operators for genetic algorithms (Stein and Bäck, 2024), thus automating components traditionally crafted by humans. While early applications in scheduling and shop floor problems have proven feasible, issues of scalability remain, especially as prompt length and complexity increase in real-world problems.

2.2.2. Integrating LLMs with evolutionary computation

To overcome the limitations of standalone LLMs, Liu et al. (2023b) proposed hybrid frameworks that integrate evolutionary computation (EC). These approaches employ EC for prompt evolution, heuristic selection, and solution refinement. For example, FunSearch evolves and filters LLM-generated programs, outperforming handcrafted heuristics in several optimization problems, though at significant computational cost (Romera-Paredes et al., 2024). Similarly, Liu et al. (2023a) introduced the EoH framework, combining LLMs with EC to discover efficient routing heuristics, while mitigating premature convergence through evolutionary guidance. Although promising, such methods typically require extensive LLM interactions, raising concerns over efficiency and scalability in complex routing scenarios.

2.2.3. Frameworks with reflection and self-improvement

Beyond static heuristic generation, reflection and self-debugging mechanisms have emerged as crucial enhancements. The ReEvo framework (Ye et al., 2024) proposes an LLM-based hyper-heuristic that integrates generation, evaluation, and reflective feedback. By comparing and refining heuristics, ReEvo improves sample efficiency and yields more reliable heuristics for combinatorial optimization. Likewise, Mont Carlo tree search-based automatic heuristic design structures LLM-generated heuristics into a search tree, enabling deeper exploration of the solution space (Li et al., 2025). Reflection frameworks further empower LLMs to debug and verify their outputs, substantially improving feasibility and optimality in VRPs.

2.2.4. Applications in transportation and logistics

Recent studies show that LLMs can act as high-level reasoning and interaction components across transportation and logistics systems. Rong et al. (2025) demonstrate that generative AI, including LLMs, has been applied to a broad spectrum of intelligent transportation tasks such as traffic management, mobility services, vehicle intelligence, and logistics decision support. Wang et al. (2023) show that LLMs can function as vehicle co-pilots, providing planning assistance, explanations, and interactive decision support in intelligent driving contexts. Wang et al. (2025) further indicate that LLMs can serve as agentic decision-makers for day-to-day route choice, producing plausible routing decisions based on contextual reasoning. These findings position routing problems as a natural and important application domain for LLM-driven methods, motivating the recent focus on automatic heuristic design for vehicle routing and related coordination problems.

LLM-driven automatic heuristic design has shown encouraging results in TSP, CVRP, and drone path planning. LLM-driven evolutionary algorithms (LMEA) achieve competitive performance on TSP with minimal domain knowledge, while hybrid approaches can directly transform natural language descriptions into executable VRP solvers (Huang et al., 2024). In drone path planning, LLMs have been combined with reinforcement learning and classical algorithms such as A* (Meng et al., 2024; Zhou et al., 2025) to guide exploration, reduce ineffective sampling, and enhance efficiency in large-scale path planning. For instance, Wu et al. (2024) incorporated LLM-generated heuristics into Q-learning (LLM-QL), improving sample efficiency and reducing ineffective exploration, albeit with increased computational costs. Other multimodal methods integrate visual encoders with temporal modeling to tackle dynamic environments, though real-time performance remains a key challenge.

Overall, LLMs have introduced a paradigm shift in heuristic design for routing problems. Serving as optimizers, designers, and reflective agents, LLMs not only automate heuristic construction previously reliant on human expertise but also introduce new mechanisms such as self-debugging and evolutionary optimization. Despite challenges in scalability, efficiency, and generalization, evidence from frameworks like FunSearch, ReEvo, ARS, and LLM-QL underscores their feasibility and promise. Future research combining hybrid methods with reflection mechanisms is expected to drive the development of more robust, adaptive, and scalable heuristic design for routing problems.

2.3. Summary of related works

The VDCRPs have gained growing attention for its potential to improve efficiency in last-mile and large-scale logistics. Existing research commonly classifies VDCRPs into SVSD, SVMD, and MVMD models, reflecting progressive complexity as the number of vehicles and drones increases. Traditional heuristic and metaheuristic approaches have provided effective solutions, but their dependence on manually designed rules constrains scalability and adaptability when facing new problem variants. Meanwhile, LLMs have emerged as promising tools for automatic routing heuristic design. By leveraging natural language interaction, code generation, and self-reflection, LLMs can autonomously generate and refine problem-solving strategies. Studies have shown that LLMs can not only produce feasible heuristics but also incorporate reflection and evolutionary mechanisms to enhance robustness and generalization.

The convergence of VDCRPs and LLM-driven automatic routing heuristic design offers a novel research direction. The structural hierarchy of SVSD, SVMD, and MVMD formulations provides a natural setting to evaluate adaptability, while the generative and reflective capacities of LLMs create opportunities for scalable and generalizable heuristic frameworks. Building on these synergies, this study is motivated to explore how LLMs can automate and enhance heuristic design for VDCRPs, bridging classical modeling insights with emerging generative AI methodologies.

3. Problem specification

This section provides the problem specification that underpins the proposed LLM-VD framework. Rather than treating the mathematical model as a standalone problem definition or a target to be solved directly, we use it to formally specify the optimization objective, feasibility conditions, and evaluation criteria that govern heuristic generation. In this sense, the problem specification serves

as a normative reference that defines what constitutes a valid and high-quality solution within the vehicle-drone collaborative routing setting.

This section focuses on the SVSD problem as a minimal yet representative coordination scenario that captures the essential interactions between vehicle routing and drone operations. Section 3.1 presents the mathematical formulation of the SVSD problem, which defines the objective and core constraints that characterize feasible coordination. More complex variants, including the SVMD and MVMD problems, extend this base specification with additional coordination constraints and are therefore deferred to Appendix A to avoid redundancy in the main text.

Building on this formulation, Section 3.2 explains how the SVSD model is operationalized into a formulation-guided task description for heuristic generation. Through this process, the mathematical specification is transformed into a structured natural-language description that defines the heuristic generation task, while model-based evaluation ensures strict adherence to the original objective and constraints. Together, Sections 3.1 and 3.2 establish a coherent link between formal problem specification and model-guided heuristic generation.

3.1. Formulation of SVSD

The SVSD problem addresses the logistics of serving a set of customers using a coordinated system of one distribution vehicle and one drone, both operating from a central depot. The primary objective is to devise a single, continuous tour that begins and ends at the depot while visiting every customer node. Each customer can be served either directly by the vehicle or by the drone in a sortie mission. A key constraint dictates that the vehicle must handle any delivery where the customer's requirements exceed the drone's operational capabilities, such as its maximum flight range (endurance) or load capacity. In this collaborative model, the drone is transported by the vehicle along its route, launched to perform a specific delivery, and subsequently returns to rendezvous with the vehicle at a designated point to continue the tour.

The SVSD model is typically described under the following assumptions:

- The tours for both the vehicle and the drone must originate and terminate at the central depot.
- Visits are restricted exclusively to customer nodes, and each customer may be served only once.
- The drone may only service one customer per sortie before needing replenishment, while the vehicle can serve multiple customers consecutively.
- In the synchronized model, all drone operations (launches and landings) must occur at designated vehicle nodes, not while the vehicle is in transit.
- In the synchronized model, all drone support operations, including charging, battery replacement, and cargo replenishment, are conducted on board the vehicle while it is stationary at a designated node.

The related notations of SVSD are given below:

Indices and sets

C set of all customer nodes $\{1, 2, \cdots, c\}$;
N set of all nodes $\{0, 1, 2, \cdots, c, c+1\}$, where 0 and $c+1$ both denote the depot;
P set of drone single-flight paths;
N_d set of drone-visited customer nodes;
N_v set of vehicle-visited customer nodes;

Parameters

$\langle i, j, k\rangle$ 3-tuple of the drone path;
τ_{ij} vehicle travel time from node i to j;
τ'_{ij} drone flight time from node i to j;
t_i vehicle arrive time at node i;
t'_i drone arrive time at node i;
e drone flight endurance;
M a sufficiently large number;

Decision variables

x_{ij} binary; equal to 1 if the vehicle travels from the node i to j, otherwise 0;
y_{ijk} binary; equal to 1 if the path of the drone is the tuple $\langle i, j, k\rangle$, otherwise 0.

$$[\mathbf{SVSD}]\mathrm{Min}\left\{\mathrm{Max}\left\{t_{c+1}, t'_{c+1}\right\}\right\} \tag{1}$$

subject to

$$\sum_{\substack{i\in N_v \\ i\neq j}} x_{ij} + \sum_{\substack{i,k\in N_v \\ i\neq j\neq k}} \sum_{\langle i,j,k\rangle\in P} y_{ijk} = 1 \forall j \in C \tag{2}$$

$$\sum_{j\in N_v} x_{0,j} = \sum_{i\in N_v} x_{i,c+1} = 1 \tag{3}$$

$$\sum_{\substack{i\in N \\ i\neq j}} \sum_{\substack{k\in N \\ \langle i,j,k\rangle\in P}} y_{ijk} \leq 1 \forall j \in N_d,\ i \neq j \neq k \tag{4}$$

$$2y_{ijk} \leq \sum_{\substack{h\in N_v \\ h\neq i}} x_{hi} + \sum_{\substack{l\in N_v \\ l\neq k}} x_{lk} \forall i,k \in \{N_v : i \neq k\}, j \in N_d \tag{5}$$

$$t_i' \geq t_i - M\left(1 - \sum_{\substack{j\in N_d \\ i\neq j}} \sum_{\substack{k\in N_v \\ \langle i,j,k\rangle\in P}} y_{ijk}\right) \forall i \in N_v \tag{6}$$

$$t_i' \leq t_i + M\left(1 - \sum_{\substack{j\in N_d \\ i\neq j}} \sum_{\substack{k\in N_v \\ \langle i,j,k\rangle\in P}} y_{ijk}\right) \forall i \in N_v \tag{7}$$

$$t_k' \geq t_k - M\left(1 - \sum_{\substack{i\in N_v \\ i\neq k}} \sum_{\substack{j\in N_d \\ \langle i,j,k\rangle\in P}} y_{ijk}\right) \forall k \in N_v \tag{8}$$

$$t_k' \leq t_k + M\left(1 - \sum_{\substack{i\in N_v \\ i\neq k}} \sum_{\substack{j\in N_d \\ \langle i,j,k\rangle\in P}} y_{ijk}\right) \forall k \in N_v \tag{9}$$

$$t_k' - \left(t_j' - \tau_{ij}'\right) \leq e + M\left(1 - y_{ijk}\right) \forall i,k \in \{N_v : i \neq k\}, j \in N_d, \langle i,j,k\rangle \in P \tag{10}$$

The SVSD problem formulation aims to minimize the overall cost of deliveries, which, in this context, corresponds to minimizing the total flight time. The objective function (1) seeks to minimize the latest time at which either the vehicle or the drone returns to the depot after completing all assigned tasks. Constraint (2) requires that every customer node is served exactly once, either by the vehicle or by the drone, while Constraint (3) ensures that the vehicle both departs from and returns to the depot. Constraint (4) prohibits the drone from visiting the same customer multiple times. Constraints (5) through (9) impose synchronization between the vehicle and the drone at joint nodes: when the drone departs from a vehicle stop at node i, delivers to node j, and rejoins the vehicle at node k, nodes i and k must also be included in the vehicle's route, and the timing of both the departure from i and the rendezvous at k must be harmonized between the vehicle and the drone; additional terms can be incorporated if takeoff, landing, or recharging times need to be considered. Finally, Constraint (10) enforces the drone's endurance limitation, typically expressed as a maximum flight time parameter e, where t_k' denotes the drone's arrival at node k and $t_j' - \tau_{ij}'$ represents the time when the drone begins its sortie from node j to serve customer i. A sufficiently large constant M is used to linearize logical constraints for Constraints (6) to (10), ensuring the validity of these constraints, particularly when dealing with conditions involving drone departures, arrivals, and flight times.

3.2. Formulation-guided task description

In this paper, the term task description refers to a structured natural-language specification derived from the SVSD formulation and directly integrated into the prompt used by LLM-VD. Its purpose is not to describe individual instances or request numerical solutions, but to define the heuristic generation task in an explicit, executable-oriented manner. As summarized in the task description box, it specifies the required inputs and outputs, states the optimization objective in operational terms (minimizing the makespan, i.e., the

latest return time of either the vehicle or the drone to the depot), and enumerates the feasibility requirements that candidate heuristics must satisfy.

Formulation-guided task description for the SVSD

Task: Design a Python heuristic for the Single-Vehicle Single-Drone (SVSD) routing problem.

Inputs: {instance}: a dictionary containing customer coordinates, customer demands, drone capacity, and drone endurance. {time_matrices}: a dictionary containing travel times for the vehicle and the drone.

Outputs: {vehicle_route: [...], drone_sorties: [(launch, customer, rendezvous), ...]}

Objective: Minimize the total completion time (makespan), defined as the latest time at which either the vehicle or the drone returns to the depot after all customers have been served.

Problem constraints:

The vehicle route must start and end at the depot.

Each customer must be served exactly once, either by the vehicle or by the drone.

A customer can be served by the drone only if its demand does not exceed the drone capacity.

The drone sortie consists of a launch from a vehicle stop, a visit to a single customer, and a rendezvous with the vehicle at another vehicle stop.

The total flight time of the drone sortie must not exceed the drone endurance.

The vehicle and the drone are synchronized at the rendezvous node; therefore, the time contribution of a segment involving the drone sortie is determined by the slower of the vehicle progression and the drone flight.

Only one drone sortie can be active at any time, and selected drone sorties must not overlap along the vehicle route.

Heuristic design rules:

To ensure simple and robust synchronization, restrict candidate drone sorties to triples of consecutive nodes along the vehicle route in forward order.

The depot may act as a launch or rendezvous node if it appears in the route, but it is never selected as a drone-served customer.

Expected heuristic framework:

Construct an initial vehicle route that visits all customers.

Scan the vehicle route and enumerate all feasible consecutive triples.

For each feasible triple, compare the time incurred when the customer is served by the vehicle with the synchronized time required when the customer is served by the drone, and compute the resulting time saving.

Rank candidate triples by time saving and greedily select non-overlapping drone sorties.

Remove drone-served customers from the vehicle route and output a coordinated solution consisting of a vehicle route and a set of drone sorties.

By grounding the task description in the formulation, heuristic generation is guided by model-defined objectives and constraints rather than ad hoc prompting. Specifically, the task description operationalizes the SVSD specification through three tightly coupled components. First, it provides a clear interface for the foundation model by fixing the expected input schema (instance information and travel-time matrices) and the output format (a vehicle route and a set of drone sorties). Second, it restates the core constraints of feasible vehicle-drone coordination in natural language, including depot start/end for the vehicle route, single-service for each customer, drone capacity and endurance limits, and synchronization at launch and rendezvous nodes. The synchronization semantics are made explicit: when the drone sortie is executed, the time contribution of the corresponding route segment is determined by the slower of the vehicle progression and the drone flight, which is consistent with a makespan objective. Third, it introduces a heuristic template that constrains how candidate drone sorties are constructed, making the search space tractable and the generated heuristics more robust in practice.

It is important to clarify that certain statements in the task description are heuristic design rules rather than modifications to the underlying SVSD definition. In particular, restricting drone sorties to triples of consecutive nodes along the vehicle route is adopted as a template to enforce simple and reliable synchronization during heuristic construction. This design choice is explicitly separated from the problem constraints in the box to avoid ambiguity: the formulation defines feasibility and performance criteria, while the template specifies how the heuristic is expected to explore feasible coordinated decisions.

Finally, consistency between the generated heuristics and the SVSD formulation is not ensured by prompting alone. All candidate heuristics produced under the task description are executed and validated by a unified evaluation module that implements feasibility checks and computes the corresponding makespan. If a candidate violates constraints or fails to execute, it is marked infeasible and accompanied by diagnostic feedback, which is then used within LLM-VD to refine subsequent candidates. Through this closed loop, the SVSD formulation remains the authoritative reference for correctness and performance, and the task description serves as the explicit interface that connects the formal model to model-guided heuristic generation.

By converting the mathematical formulation into a structured natural-language representation and coupling it with strict evaluation and feedback, LLM-VD enables automated heuristic generation without sacrificing rigor. This mechanism also provides a clear foundation for extending the framework to more complex VDCRPs, such as SVMD and MVMD, which are presented in Appendix A.

4. Methodology: LLM-VD

This section presents the proposed LLM-VD methodology for automatic heuristic design tailored to VDCRPs. Built upon the EoH paradigm, LLM-VD evolves executable heuristics under a unified interface and leverages evaluation-driven feedback to improve feasibility and performance across generations. We first introduce the overall workflow of LLM-VD in Section 4.1. We then describe the integrated heuristics evaluation and self-debugging module in Section 4.2, including how failure feedback is collected and subsequently utilized for refinement. Finally, Section 4.3 details the prompt strategies used to instantiate the evolutionary operators for VDCRPs.

4.1. Algorithmic framework of LLM-VD

Inspired by the EoH paradigm, we propose LLM-VD, a self-debugging LLM-driven evolutionary framework for the automatic design of heuristics for VDCRPs. Unlike one-shot heuristic generation, LLM-VD treats the LLM as an iterative generator whose outputs are continuously refined through evaluation feedback and evolutionary selection.

As formalized in Algorithm 1 and illustrated in Fig. 1, LLM-VD maintains a population P of executable heuristics and iteratively improves them over multiple generations. Each heuristic in LLM-VD is represented by a pair consisting of a high-level linguistic description (referred to as a thought) and a corresponding executable implementation. Specifically, the thought is a concise natural-language description that summarizes the core idea and decision logic of the heuristic, while the implementation is a Python function generated by the LLM that follows a predefined interface, enabling systematic evaluation and comparison. This design allows LLM-VD to separate high-level heuristic ideas from their executable implementations, providing more flexibility for evolutionary refinement.

Step 1: Population initialization

Given a task description T, which specifies the statement of a VDCRP variant and the required heuristic framework, LLM-VD initializes the population by repeatedly invoking the initialization prompt **I1(Initialization)**. Each generated heuristic is immediately evaluated, and only those that successfully pass the evaluation are admitted into the initial population until the population size reaches N. This ensures that all individuals in P are executable and feasible from the outset.

Step 2: New heuristic generation by evolution operators

At each generation g, LLM-VD generates new candidate heuristics using four EoH-style evolution operators: **E1(Exploration 1)**, **E2(Exploration 2)**, **M1(Modification 1)**, and **M2(Modification 2)**. Operators **E1** and **E2** generate heuristics based on two parent heuristics selected from the current population (via tournament selection) to encourage exploration, whereas **M1** and **M2** refine a single heuristic selected using inverse-rank weighted selection to promote exploitation. These operators act primarily at the thought level, where the LLM synthesizes, combines, or modifies high-level heuristic logic based on selected parent heuristics, and subsequently generates a consistent implementation under a predefined template. The detailed prompt designs for these operators are presented in Section 4.3.

Step 3: Evaluation and self-debugging

Each new candidate heuristic is evaluated independently using a unified evaluation procedure (shown in Algorithm 2), which returns a triplet $(status, score, feedback)$. Here, *status* is a binary indicator specifying whether the heuristic successfully passes all evaluation instances, *score* is the performance metric used to assess the heuristic and is computed as the negative of its average objective value over the evaluation instances, and *feedback* provides a structured description of the failure feedbacks when the heuristic fails the evaluation (e.g., runtime errors or constraint violations). A heuristic is considered successful only if it passes all evaluation instances. Failed heuristics provide informative failure feedback, which is subsequently exploited by LLM-VD through two alternative self-debugging modes.

As shown in the red dashed box in Fig. 1, two different self-debugging mechanisms can be activated depending on the mode indicator E. When the $E = 1$, failure feedback from the current generation is accumulated and injected into subsequent evolutionary prompts, guiding the LLM to avoid recurring errors. When $E = 2$, a failed heuristic is selected via inverse-rank weighted selection and repaired using the repair operator **R1(Repair)**, which prompts the LLM with the task description, the failed heuristic, and its associated feedback to generate a corrected version. Repaired heuristics are re-evaluated and incorporated into the population only if they pass the evaluation.

Step 4: Population management and termination

After evaluation and optional self-debugging, all successful heuristics are merged into the population. If the population size exceeds N, a truncation procedure retains the top-performing heuristics according to their fitness scores. This process repeats until the

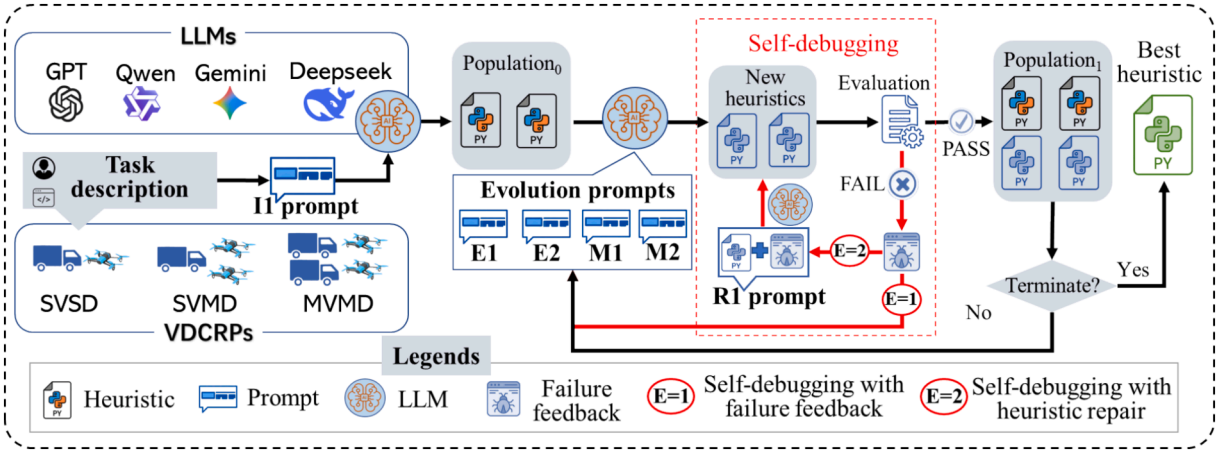


Fig. 1. The framework of LLM-VD.

maximum number of generations is reached. The final output is the best heuristic in the population, defined as the one with the highest fitness score h_{best}.

Overall, the distinguishing feature of LLM-VD lies in its feedback-driven self-debugging mechanism, which systematically transforms heuristic failures into actionable signals for refinement. This design substantially improves the robustness and efficiency of LLM-based heuristic design for constraint-rich VDCRP variants.

Algorithm 1: LLM-VD with self-debugging.

Input: population size N, number of generations G, task description T, self-debugging mode E
Output: best heuristic h_{best}
initialize $P \leftarrow \emptyset$, $g = 0$, $selfdebug_{info} \leftarrow \emptyset$
build the initial prompt $\mathbf{I1} \leftarrow T$
while $|P| < N$ **do**
  $h_{new}(desc, code) \leftarrow$ GenerateHeuristic $(\mathbf{I1})$
  $h_{new}(status, score, feedback) \leftarrow$ **Evaluation** (h_{new})
  if $status =$ PASS **then**
    $P \leftarrow P \cup \{h_{new}\}$
  end if
end while
while $g \leq G$ **do**
  $parents \leftarrow$ Tournament selection (P, N)
  $individual \leftarrow$ Inverse-rank weighted selection (P, N)
  # E1/E2 use two parents; M1/M2 use a single individual
  build four evolution prompts $\{\mathbf{E1}, \mathbf{E2}, \mathbf{M1}, \mathbf{M2}\} \leftarrow (T, selfdebug_{info})$
  $h_{e1} \leftarrow$ GenerateHeuristic $(\mathbf{E1}, parents)$
  $h_{e2} \leftarrow$ GenerateHeuristic $(\mathbf{E2}, parents)$
  $h_{m1} \leftarrow$ GenerateHeuristic $(\mathbf{M1}, individual)$
  $h_{m2} \leftarrow$ GenerateHeuristic $(\mathbf{M2}, individual)$
  $h_{can} \leftarrow (h_{e1}, h_{e2}, h_{m1}, h_{m2})$
  for each $h \in h_{can}$ **do**
    $h(status, score, feedback) \leftarrow$ **Evaluation** (h)
    Store $(h, status, score, feedback)$
  end for
  $P \leftarrow P \cup \{h_{can} | h_{can}(status) = \text{PASS}\}$
  if any $h \in h_{can}$ has $status =$ FAIL **then**
    if $M = 1$ **then**
      $selfdebug_{info} \leftarrow selfdebug_{info} \bigcup \{h(feedback) | h \in h_{can}, h(status) = \text{FAIL}\}$
    elseif $M = 2$
      $h_{fail} \leftarrow$ Inverse-rank weighted selection $\{h | h \in h_{can}, h(status) = \text{FAIL}\}$
      build repair prompt $\mathbf{R1} \leftarrow (T, h_{fail}(feedback))$
      $h_{repair} \leftarrow$ GenerateHeuristic $(\mathbf{R1}, h_{fail})$
      $h_{repair}(status, score, feedback) \leftarrow$ **Evaluation** (h_{repair})
      $P \leftarrow P \cup \{h_{repair} | h_{repair}(status) = \text{PASS}\}$
    end if
  end if
  if $|P| > N$ **then** $P \leftarrow$ TopSelection (P, N) **end if**
  $g \leftarrow g + 1$
end while
return $h_{best} \in P$ with the max score

4.2. Heuristics evaluation and self-debugging

Since evaluation outcomes determine both selection and subsequent refinement, Section 4.2 introduces the unified evaluation-and-debugging module used throughout LLM-VD. In particular, we define the evaluation interface and the failure feedback that are consistently used across VDCRP variants and throughout the evolutionary process.

4.2.1. Heuristics evaluation and failure feedback collection

We first present the heuristic evaluation procedure in Algorithm 2, which evaluates each generated heuristic on a set of instances and returns $\{status, score, feedback\}$. For each heuristic, the evaluator executes it on all evaluation instances and processes the resulting solutions through a unified validation-and-scoring routine. Specifically, *status* indicates whether the heuristic passes all evaluation instances, *score* is computed as the negative average objective value over the evaluation instances to measure heuristic performance, and *feedback* records structured failure information when evaluation is not passed.

At the core of Algorithm 2 is an abstract procedure, denoted as **FailureCheckandScore**, which encapsulates both feasibility verification and objective computation. For each instance, this procedure either confirms solution validity and returns the corresponding objective value, or detects a failure and produces a structured failure message. A heuristic is considered successful only if it

passes the evaluation on all instances; otherwise, its *status* marked as FAIL, its *score* is set to $-inf$, and the evaluator aggregates the collected failure messages into a unified feedback summary. This all-or-nothing evaluation strategy emphasizes robustness and ensures that only consistently feasible heuristics are admitted into the population.

The *feedback* component serves as the core information source for the self-debugging mechanism. By collecting descriptions of failed heuristics and aggregating their corresponding failure feedback, the framework enables the LLM to analyze and reflect on recurring error patterns, thereby reducing similar failures in subsequent heuristic generation. Since the self-debugging mechanism is intended to be applicable across different VDCRP variants, failure feedback must be derived from the evaluation of each problem type. While the overall evaluation skeleton remains identical, the concrete failure detection logic inside **FailureCheckandScore** is tailored to the specific characteristics of each variant. In this way, the evaluation module provides problem-specific guidance while preserving a unified, LLM-driven iterative refinement process.

Algorithm 2: Evaluation procedure of heuristic.

Input: heuristic h, evaluation instances I
Output: $\{status, score, feedback\}$ of h
$total_{obj} \leftarrow 0$; $valid_{cnt} \leftarrow 0$; $feedback \leftarrow \varnothing$
for each instance $i \in I$ **do**
  $S \leftarrow$ RunHeuristic (h, i)
  $(S_{valid}, S_{obj}, S_{fb}) \leftarrow$ **FailureCheckandScore** (S)
  If S_{valid} = True **then**
    $total_{obj} \leftarrow total_{obj} + S_{obj}$; $valid_{cnt} \leftarrow valid_{cnt} + 1$
  else
    $feedback \leftarrow feedback \cup \{S_{fb}\}$
  end if
end for
if $\text{valid}_{\text{cnt}} < |I|$
  return $\{\text{FAIL}, -inf, \text{Aggregate}(feedback)\}$
else
  return $\{\text{PASS}, -total_{obj}/valid_{cnt}, \varnothing\}$
end if

The heuristic evaluation procedures for the SVSD, SVMD, and MVMD problems are described in detail below, with specific failure types and corresponding feedback summarized in Tables 1–3. Across all three variants, solutions must satisfy a common set of structural requirements, including a correct return format (i.e., a valid dictionary containing vehicle routes and drone sorties), valid vehicle route structure (routes must begin and end at the depot), and customer coverage without duplication (each customer is served exactly once). At the drone operation level, all variants must comply with capacity and endurance constraints. In addition, failures in objective function computation and runtime exceptions are treated as general indicators of computational validity and algorithmic robustness. Building on this shared foundation, each problem variant further enforces additional constraints that reflect its specific operational characteristics and complexity.

For the SVSD problem (shown in Table 1), its uniqueness lies in the simplification of details. With only one vehicle and one drone, their identities are implicit, and the sortie format is a simple triplet: (launch, customer, rendezvous). The validation logic is also straightforward, requiring only that launch/rendezvous nodes exist on the single vehicle route.

In contrast, the SVMD problem (shown in Table 2) manages multiple drones from a single platform. Consequently, its sortie format expands to a quadruple, (drone_id, launch, customer, rendezvous), to distinguish between drones. It also introduces a unique failure type: “Excess sorties per node,” a multi-resource constraint preventing a node from dispatching more drones than are available.

As the most complex variant, MVMD (shown in Table 3) adds constraints for multi-vehicle management. Its sortie format, (vehicle_id, launch, customer, rendezvous), links each drone to its parent vehicle. Furthermore, it introduces two new failure types: “Vehicle number mismatch” (to align route count with fleet size) and “Vehicle capacity violation” (to check vehicle-specific loads).

Table 1
Failure feedback for SVSD heuristic evaluation failure.

Failure Type	Feedback
Return format error	The returned solution is not a valid dictionary with ‘vehicle_route’ and ‘drone_sorties’.
Vehicle route error	Vehicle route must start and end at depot 0.
Customer coverage error	Customer coverage error. Missing: **{}**, Extra: **{}**
Duplicate visits	Some customers are visited more than once.
Sortie format error	Sortie must be (launch, customer, rendezvous).
Sortie sequence error	Invalid sortie {sortie}: rendezvous must appear after launch.
Drone launch/rendezvous error	Sortie has launch/rendezvous node not on vehicle route.
Drone capacity violation	Customer {j} demand exceeds drone capacity.
Drone endurance violation	Sortie (launch, customer, rendezvous) exceeds drone endurance.
Objective calculation failure	Objective calculation resulted in infinity (likely invalid route).
Runtime exception	Function crashed during runtime.

Table 2
Failure feedback for SVMD heuristic evaluation failure.

Failure Type	Feedback
Return format error	The returned solution is not a valid dictionary with 'vehicle_route' and 'drone_sorties'.
Vehicle route error	Vehicle route must start and end at depot 0.
Customer coverage error	Customer coverage error. Missing: {}, Extra: {}
Duplicate visits	Some customers are visited more than once.
Sortie format error	Sortie must be (drone_id, launch, customer, rendezvous).
Sortie sequence error	Invalid sortie {sortie}: rendezvous must be the same node as launch.
Launch/rendezvous error	Launch node not in vehicle route or depot assigned as drone customer.
Drone capacity violation	Customer {c} demand exceeds drone capacity.
Drone endurance violation	Sortie (drone_id, launch, customer, rendezvous) exceeds drone endurance.
Objective calculation failure	Objective calculation resulted in infinity (likely invalid route).
Runtime exception	Function crashed during runtime.
Excess sorties per node	Node {c} assigned {k} sorties, exceeds num_drones={d}.

Table 3
Failure feedback for MVMD heuristic evaluation failure.

Failure Type	Feedback
Return format error	The returned solution is not a valid dictionary with 'vehicle_routes' and 'drone_sorties'.
Vehicle route error	Vehicle {v} route must start and end at depot 0.
Customer coverage error	Customer coverage error. Missing: {}, Extra: {}
Duplicate visits	Some customers are visited more than once.
Sortie format error	Sortie must be (vehicle_id, launch, customer, rendezvous).
Sortie sequence error	Invalid sortie {sortie}: rendezvous must appear after launch.
Launch/rendezvous error	Launch or rendezvous not in vehicle {v} route or depot assigned as drone customer.
Drone capacity violation	Customer {c} demand exceeds drone capacity.
Drone endurance violation	Sortie {sortie} exceeds drone endurance.
Objective calculation failure	Objective calculation resulted in infinity.
Runtime exception	Function crashed during runtime.
Vehicle number mismatch	Number of vehicle routes does not match number of vehicles.
Vehicle capacity violation	Vehicle {v} load exceeds capacity.

These constraints collectively represent the leap in complexity from single-vehicle optimization to the resource management of an entire fleet.

4.2.2. *Self-debugging mechanism based on failure feedback*

Based on the failure feedback collected during heuristic evaluation, LLM-VD incorporates a self-debugging mechanism to improve the feasibility, stability, and efficiency of LLM-generated heuristics for complex VDCRP variants. As illustrated in Fig. 2 and Fig. 3, this mechanism operates in two alternative modes that correspond to the two self-debugging settings in Algorithm 1. Both modes embed evaluation signals back into the heuristic generation loop, but they differ fundamentally in the level at which feedback is exploited, the amount of information injected into prompts, and the resulting computational overhead. To avoid ambiguity, we emphasize that $E = 1$ and $E = 2$ are mutually exclusive and are not activated simultaneously within the same generation.

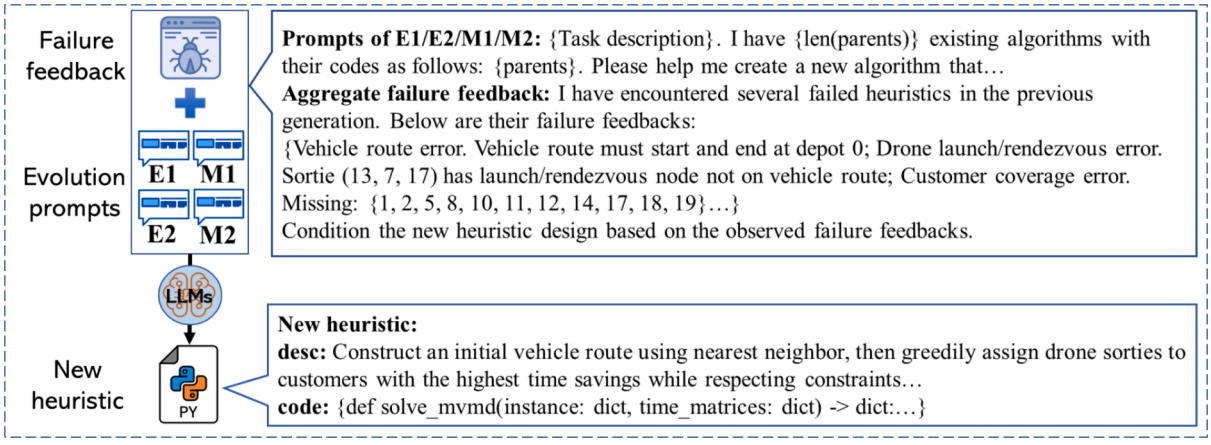


Fig. 2. Self-debugging mechanism with failure feedback.

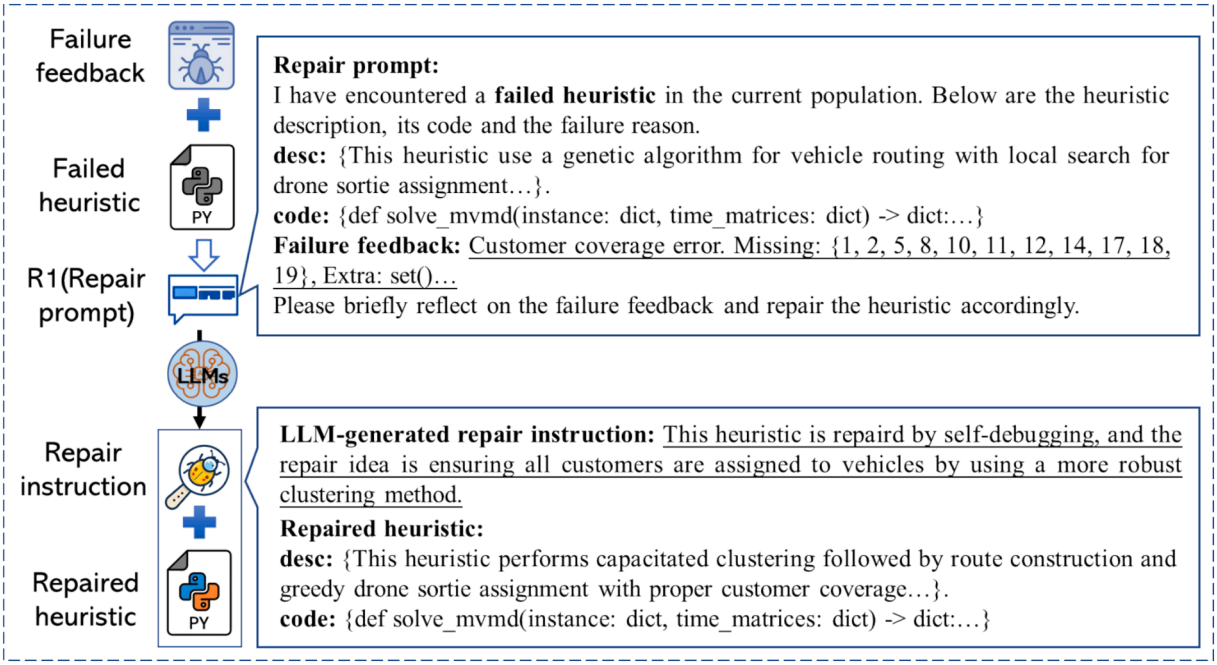


Fig. 3. Self-debugging mechanism with heuristic repair.

In the first mode ($E = 1$), the framework aggregates structured failure feedback from all failed heuristics in the previous generation and injects this summary into the evolutionary prompts used by operators **E1, E2, M1,** and **M2** in the current generation (Fig. 2). The injected feedback captures recurring failure patterns, including return-format violations, vehicle-route errors, customer-coverage defects, drone capacity or endurance violations, and runtime exceptions, while explicitly excluding the source code of failed heuristics. As a result, this mode introduces only a marginal increase in prompt length and imposes low engineering complexity and runtime overhead. Conceptually, this mode provides prompt-level feedback regularization: by conditioning the LLM on previously observed failure modes, it discourages the regeneration of known pitfalls and improves the overall feasibility rate, while preserving diversity in heuristic design and avoiding dependence on specific failed implementations.

While $E = 1$ is effective at suppressing repeated errors, it does not exploit the structural information embedded in failed heuristics that are close to feasibility. To better reuse such partial designs, LLM-VD introduces a second self-debugging mode based on explicit heuristic repair ($E = 2$), illustrated in Fig. 3. In this mode, a single failed heuristic is selected per generation using inverse-rank weighted selection among failed candidates, prioritizing near-feasible designs while keeping the repair cost bounded. The selected heuristic's high-level thought description, source code, and associated failure feedback are combined into a dedicated repair prompt (operator **R1**). The LLM is instructed to generate an LLM-generated repair instruction that directly targets the identified violations and to output a revised heuristic implementation accordingly. The repaired heuristic is then re-evaluated under the same evaluation procedure, and only those that pass all evaluation instances are admitted into the population.

Because $E = 2$ explicitly includes failed code artifacts, it typically produces substantially longer prompts and incurs higher runtime cost than $E = 1$. However, by enabling code-level, targeted correction within the current generation, this mode allows LLM-VD to recover and refine promising but imperfect designs instead of discarding them. Consequently, the second mode is expected to further improve heuristic design success rates and sample efficiency, especially for complex VDCRP variants with tight coupling between routing and drone-assignment constraints.

In summary, the proposed two self-debugging mechanisms operate at different levels. The first mode provides lightweight, generation-level guidance by injecting aggregated failure patterns, offering low-overhead robustness gains and scalability across generations. The second mode, in contrast, leverages concrete failed artifacts to perform targeted algorithmic revision, delivering stronger corrective power at the cost of higher prompt length and runtime. By restricting $E = 2$ to at most one repaired heuristic per generation via inverse-rank weighted selection, LLM-VD prevents excessive computational overhead while still benefiting from deep reuse of near-feasible designs. LLM-VD treats $E = 1$ and $E = 2$ as alternative, mutually exclusive configurations consistent with Algorithm 1: the first self-debugging mode emphasizes efficiency and stability through feedback conditioning, whereas the second self-debugging mode emphasizes corrective strength and sample efficiency through explicit heuristic repair.

4.3. Prompt strategies

This section details the prompt strategies that instantiate the six operators **{I1, E1, E2, M1, M2, R1}** for VDCRPs. These prompts encode the task description, heuristic interface, parent heuristics, and (when enabled) failure feedback into structured inputs to guide

the LLM in producing executable heuristic code. A key methodological principle underlying all prompt designs is that the LLM is explicitly required to first generate a one-sentence high-level description of the heuristic logic (i.e., a "thought"), followed by an executable implementation under a predefined template. This explicit separation between heuristic logic and code implementation enforces a clear abstraction boundary, which facilitates evolutionary recombination at the logic level, supports effective self-debugging through feedback, and enables reliable and consistent evaluation of generated heuristics. While the prompts in this study are designed for VDCRPs, the overall LLM-VD framework is not restricted to VDCRPs and can be transferred to other combinatorial optimization problems by revising the task description and operator-specific prompt templates accordingly.

In the proposed LLM-VD framework, prompts follow a structured design consisting of five key components: (1) task description, (2) parent heuristic(s), (3) failure feedback, (4) expected output, and (5) notes. The role of each component is described below.

- **Task description:** It provides a natural language specification of the problem, tailored to the characteristics of the three vehicle-drone collaborative routing variants (SVSD, SVMD, and MVMD). It defines the optimization objectives, the core operational constraints, and the heuristic design framework, thereby ensuring that the LLM aligns with the intended problem-solving goals. An example of the task description prompt for SVSD is given in section 3.2.
- **Parent heuristic(s):** This component supplies the descriptions and Python implementations of existing heuristics, enabling in-context learning and further innovation by the LLM. The number of parent heuristics depends on the operator: initialization prompts (I1) contain none, exploration prompts (E1 and E2) involve multiple heuristics, and mutation prompts (M1 and M2) are based on a single heuristic.
- **Failure feedback:** This component provides categorized bug information from heuristics that fail unit tests, serving as a key element of the self-debugging mechanism. It can be incorporated into the prompts of evolution operators E1, E2, M1, and M2 to help the LLM avoid similar errors, or combined with a failed heuristic as the repair prompt R1 to guide its correction. The failure types and corresponding feedback for different VDCRP variants are detailed in Section 4.2.1.
- **Expected output:** It explicitly requires the LLM to provide a concise one-sentence description of the new heuristic within a box **{}**, followed by a Python function implementing the predefined interface according to **{template_program}**. This standardized function template ensures that the generated heuristics can be executed and automatically evaluated within the framework. The function template prompt **{template_program}** for SVSD is provided in Appendix B.
- **Notes:** Some additional instructions are provided to restrict unnecessary explanations, thereby improving the efficiency and usability of the generated heuristics.

The six prompt strategies derived from the integration of the aforementioned predefined components are introduced in detail as follows:

- **I1** is used to generate the initial population. It relies solely on the task description and produces entirely new heuristics without parent heuristics or failure feedback. The prompt is formulated as follows: "**{task_description}**. First, describe your new algorithm and main steps in one sentence. The description must be inside within boxed **{}**. Next, implement the following Python function: **{template_program}**. Do not give additional explanations."
- **E1** is designed for large-scale exploration. Based on the task description and multiple parent heuristics, the LLM is asked to create heuristics that are fundamentally different from existing ones. Failure feedback may be optionally included to avoid repeated failures. The prompt is formulated as follows: "**{task_description}**. I have **{len(parents)}** existing algorithms with their codes as follows: **{parents_code}**. Please help me create a new algorithm that has a totally different form from the given ones. First, describe your new algorithm and main steps in one sentence. The description must be inside within boxed **{}**. Next, implement the following Python function: **{template_program}**. Do not give additional explanations. **{failure_feedback}**."
- **E2** is geared toward small-scale exploration. The LLM is instructed to identify the shared backbone ideas among parent heuristics and design a new heuristic inspired by them. Compared with E1, E2 emphasizes incremental innovation. Failure feedback may also be included. The prompt is formulated as follows: "**{task_description}**. I have **{len(parents)}** existing algorithms with their codes as follows: **{parents_code}** Please help me create a new algorithm that has a totally different form from the given ones but can be motivated from them. Firstly, identify the common backbone idea in the provided algorithms. Secondly, based on the backbone idea describe your new algorithm in one sentence. The description must be inside within boxed {}. Thirdly, implement the following Python function: **{template_program}**. Do not give additional explanations. **{failure_feedback}**."
- **M1** is applied to a single parent heuristic for structural modification. The LLM must generate a functionally improved but structurally different version, focusing on altering algorithmic steps or logic. Failure feedback can be incorporated to guide the mutation. The prompt is formulated as follows: "**{task_description}**. I have one algorithm with its code as follows: **{parent_code}**. Please assist me in creating a new algorithm that has a different form but can be a modified version of the algorithm provided. First, describe your new algorithm and main steps in one sentence. The description must be inside within boxed **{}**. Next, implement the following Python function: **{template_program}**. Do not give additional explanations. **{failure_feedback}**."
- **M2** is also applied to a single parent heuristic, but focused on parameter-level changes. The LLM identifies key algorithmic parameters and adjusts them to enhance performance, emphasizing parameter tuning rather than structural redesign. Failure feedback may be added. The prompt is formulated as follows: "**{task_description}**. I have one algorithm with its code as follows: **{parent_code}**. Please identify the main algorithm parameters and assist me in creating a new algorithm that has a different parameter setting of the score function provided. First, describe your new algorithm and main steps in one sentence. The

description must be inside within boxed **{}**. Next, implement the following Python function: **{template_program}**. Do not give additional explanations. **{failure_feedback}**."

- **R1** is designed for the automatic repair of a selected failed heuristic from the current generation. Based on its specific code and the failure feedback obtained from unit tests, the LLM is instructed to repair the failed heuristic. The prompt is as follows: "**{task_description}**. I have encountered a failed heuristic in the current population. Below are the heuristic description, its code and the failure reason: **{failed_heuristics}** and **{failure_feedback}**. Please briefly reflect on the failure feedback and repair the heuristic accordingly. Do not output your reflection. After reflection, carefully repair the failed heuristic. Give the repaired heuristic as follows: 1. Briefly describe the heuristic and main steps in one sentence. 2. Then, implement the Python function according to the following template: **{str(temp_func)}**. Do not give any additional explanations."

5. Numerical experiments

To evaluate the effectiveness of the proposed LLM-VD framework in generating heuristics for VDCRPs, we conduct a series of numerical experiments in this section. We first present the instances and parameter settings used for testing the proposed LLM-VD framework. The experiments consist of three main parts. First, we report the best-performing heuristics and corresponding results obtained by LLM-VD on three types of VDCRPs, and further compare LLM-VD with a baseline heuristic. Subsequently, we provide a comparative analysis of the experimental results across different LLMs and assess the benefit of the proposed self-debugging mechanism. Finally, we extend the study to a real-world case with a drone recharging constraint. All numerical experiments were conducted on a workstation equipped with an Intel Xeon Gold 5218R 2.10 GHz CPU, with the code implemented in Python 3.12 using PyCharm 2025.2.

5.1. Instances and parameters setting

To facilitate a rigorous and reproducible evaluation of algorithmic performance, a systematic methodology was employed to generate 8 test instances for each of the SVSD, SVMD, and MVMD problem classes. All instances were built upon a common foundational framework, each comprising pre-defined scale of nodes with coordinates randomly generated within a $[0, 1]^2$ unit square. Node 0 was consistently designated as the depot with zero demand. To simulate realistic operational environments, asymmetric distance metrics were adopted for vehicle and drone movement: vehicles adhere to the Manhattan distance to represent travel along a grid-like urban network, while drones utilize the Euclidean distance to reflect direct, point-to-point flight paths. Building on this foundation, the testing instances for SVSD problem featured a single vehicle with effectively infinite capacity and one drone with a cargo capacity of 5 units and a flight endurance of 1.0. For these instances, customer demands were randomly generated as integers within the range of $[1, 10]$. The SVMD instances served as a direct extension, increasing the drone contingent to three while retaining all other parameters. In contrast, the generation logic for the MVMD instances was the most complex, defining a fleet of three vehicles, each with a finite capacity of 50. To ensure that each instance necessitated multi-vehicle cooperation, a sophisticated demand generation mechanism strictly constrained the total customer demand to the range of $[100, 150]$, guaranteeing that the total demand

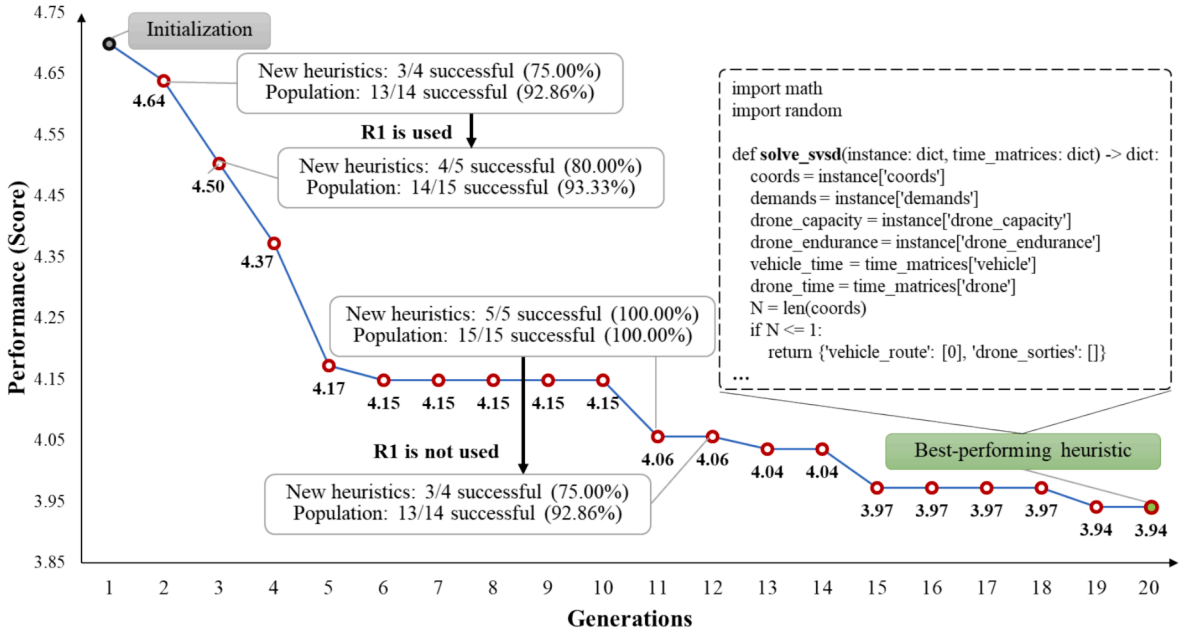


Fig. 4. Evolution of LLM-VD ($E = 2$) for SVSD problem.

would exceed the capacity of any single vehicle while remaining serviceable by the entire fleet. Finally, to ensure the reproducibility of the experiments, the entire instance generation process was governed by fixed random seeds.

In our experiments, the key parameters for the evolutionary framework were configured as follows. The population size was set to 10, and at the end of each generation, a survival selection mechanism based on an elitist strategy was employed to ensure that the top N individuals from the current generation advanced directly to the next, thereby promoting algorithmic convergence. The generation of new heuristics was driven by four genetic operators, wherein crossover-type operators (E1 and E2) operated on two parents, while mutation-type operators (M1 and M2) operated on a single parent. To accelerate the population evaluation, the entire process utilized 5 parallel evaluator threads. Finally, the evolutionary process was set to terminate when either of two conditions was met: a maximum of 20 generations had been completed, or a total of 200 samples had been evaluated.

To evaluate the performance of LLM-VD in generating heuristics using different LLMs, this study employs four representative LLMs released as of September 2025 for the heuristics design of VDCRPs: DeepSeek (model version: *deepseek-chat*), Qwen (model version: *qwen3*-max), Gemini (model version: *gemini-2.5-flash*), and ChatGPT (model version: *gpt-5-mini*). It is worth noting that the models selected are not necessarily the most powerful versions currently available for each LLM. For example, DeepSeek has also released *deepseek-reasoner*, and ChatGPT offers the full version of *gpt-5*, both of which are equipped with enhanced reasoning capabilities and theoretically superior performance. However, preliminary tests revealed that these reasoning-focused versions incur significantly higher computational costs and are prone to diverging from the original task specification during extended reasoning, which, in turn, leads to a sharp increase in heuristic generation failures. Balancing efficiency and performance, we therefore opted for the aforementioned model versions in this study. Finally, the quality of the heuristics generated is evaluated by ***Score***, defined as the average objective function value across 8 test instances. A lower score indicates a higher-quality heuristic.

5.2. Results

5.2.1. Best-performing heuristics generated by LLM-VD

This section first examines the best-performing heuristics generated by the proposed LLM-VD framework for the three classes of vehicle-drone routing problems. The results demonstrate that LLM-VD is able to automatically design effective heuristic algorithms for complex cooperative routing problems by exploiting problem-specific prompting strategies. The Python implementations of the final heuristics for the SVSD, SVMD, and MVMD problems are provided in Appendices C, D, and E, respectively.

Fig. 4 illustrates the heuristic evolution process of LLM-VD ($E = 2$) for the SVSD problem. Each generation corresponds to one iteration of heuristic design, evaluation, and selection, during which new heuristic variants are produced through evolutionary operators and assessed on the training instances. Fig. 4 shows a clear monotonic improvement trend in the best objective value, indicating that LLM-VD is able to progressively refine heuristic structures rather than relying on isolated random improvements. Early generations exhibit relatively large performance gains, while later generations display smaller but consistent refinements, reflecting a typical transition from exploration to exploitation.

In addition to performance evolution, Fig. 4 reports the success rate of heuristics generation for both newly generated heuristics and

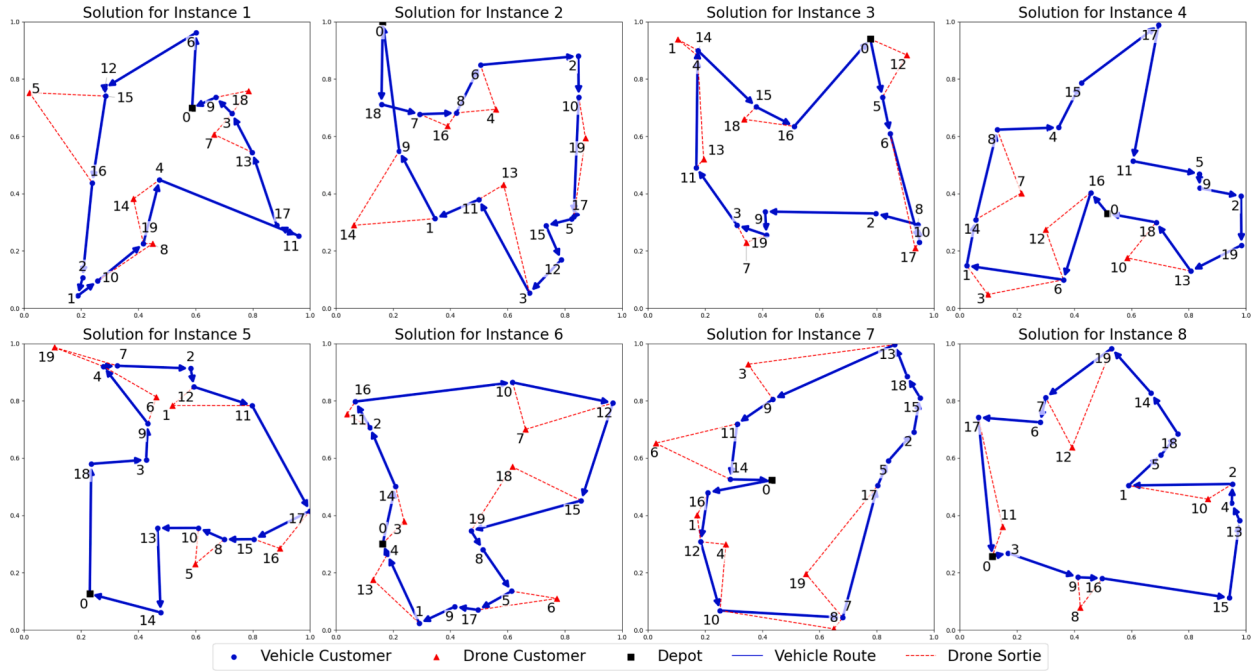


Fig. 5. Solutions obtained by the best-performing heuristic for the SVSD instances.

the entire population at each generation. The success rate of heuristics generation is computed by counting the number of heuristics that successfully pass the evaluation procedure in each generation. The overall population consistently maintains a high feasibility rate, often reaching 100%, which indicates that feasible heuristic structures are effectively preserved throughout the evolutionary process. The success rate of newly generated heuristics shows greater variability, highlighting the exploratory nature of the evolutionary operators. Notably, the repair operator **R1** is activated adaptively only when the previous population's feasibility is incomplete (e.g., from the 2nd to the 3rd generation), reinforcing feasibility when needed, without restricting the search once stable heuristic structures have emerged. This adaptive mechanism allows LLM-VD to balance robustness and innovation during heuristic evolution.

After presenting the overall evolution process of LLM-VD, we next describe in detail the core ideas and procedural steps of the best-performing heuristic generated for the three VDCRP variants. For the SVSD problem, the best-performing heuristic is generated by ChatGPT. The method first constructs a high-quality single-vehicle route and then incorporates drone sorties to reduce overall travel time. It creates multiple diversified seed routes through three constructive strategies, namely nearest-neighbor biased selection, farthest-biased selection, and random selection. Each route is refined using a 2-opt local search procedure. For every candidate route, the algorithm scans consecutive triples of nodes to identify feasible launch, service, and rendezvous patterns that satisfy drone capacity and endurance constraints. Feasible sorties are evaluated according to their time savings, and a greedy selection procedure chooses a set of non-overlapping sorties. Customers served by drones are removed from the vehicle route, and the combined vehicle-drone solution achieving the lowest adjusted travel time is selected. The corresponding cooperative routing result is illustrated in Fig. 5.

For the SVMD problem, the best-performing heuristic is generated by Qwen. The method first identifies customers who must be served by the vehicle due to drone capacity or endurance limits, and treats them as initial medoids. It then adopts a k-medoids style expansion that selects additional vehicle centers using a score balancing drone time savings and vehicle detour cost. Remaining drone-eligible customers are assigned to feasible centers according to a drone-time based scoring rule. A nearest-neighbor procedure constructs the initial vehicle route, which is refined through 2-opt local search. After route improvement, customer-center assignments are updated, ensuring feasibility under drone limits. Finally, each center dispatches its available drones to serve assigned customers, and the algorithm outputs the refined vehicle route and the full set of drone sorties. The corresponding vehicle-drone cooperative routing result is illustrated in Fig. 6.

For the MVMD problem, the best-performing heuristic is also generated by Qwen. The method begins with a sweep-based initialization that assigns customers to vehicles while respecting capacity limits, followed by nearest-neighbor route construction for each vehicle. A 2-opt refinement is applied to improve route quality. For every vehicle route, the algorithm evaluates all feasible launch, service, and rendezvous combinations to identify drone sorties that satisfy capacity and endurance constraints. Each sortie is assessed by its time saving relative to vehicle detours, and a greedy non-overlapping selection procedure determines the final set of assigned drone tasks. The algorithm then removes drone-served customers from vehicle routes and produces an initial multi-vehicle cooperative solution. A subsequent inter-vehicle local search reassigns customers between vehicles whenever capacity allows and yields a shorter combined routing cost. The resulting multi-vehicle and multi-drone coordinated solution is returned as the final output. The resulting solution is shown in Fig. 7.

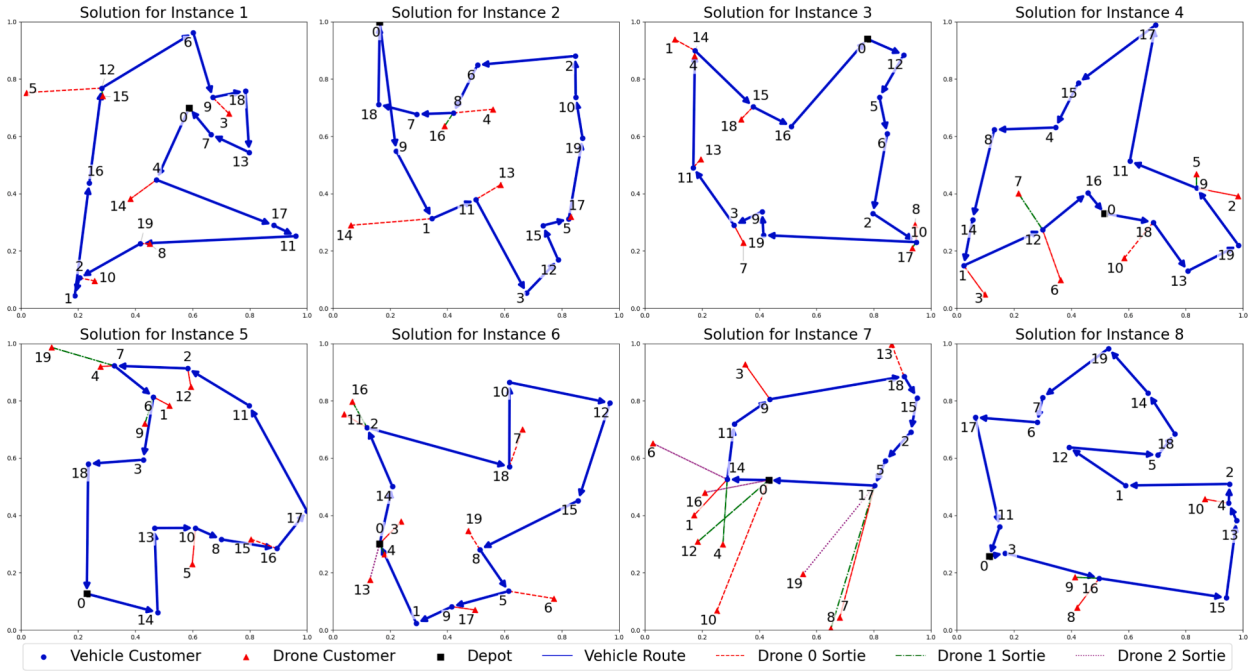


Fig. 6. Solutions obtained by the best-performing heuristic for the SVMD instances.

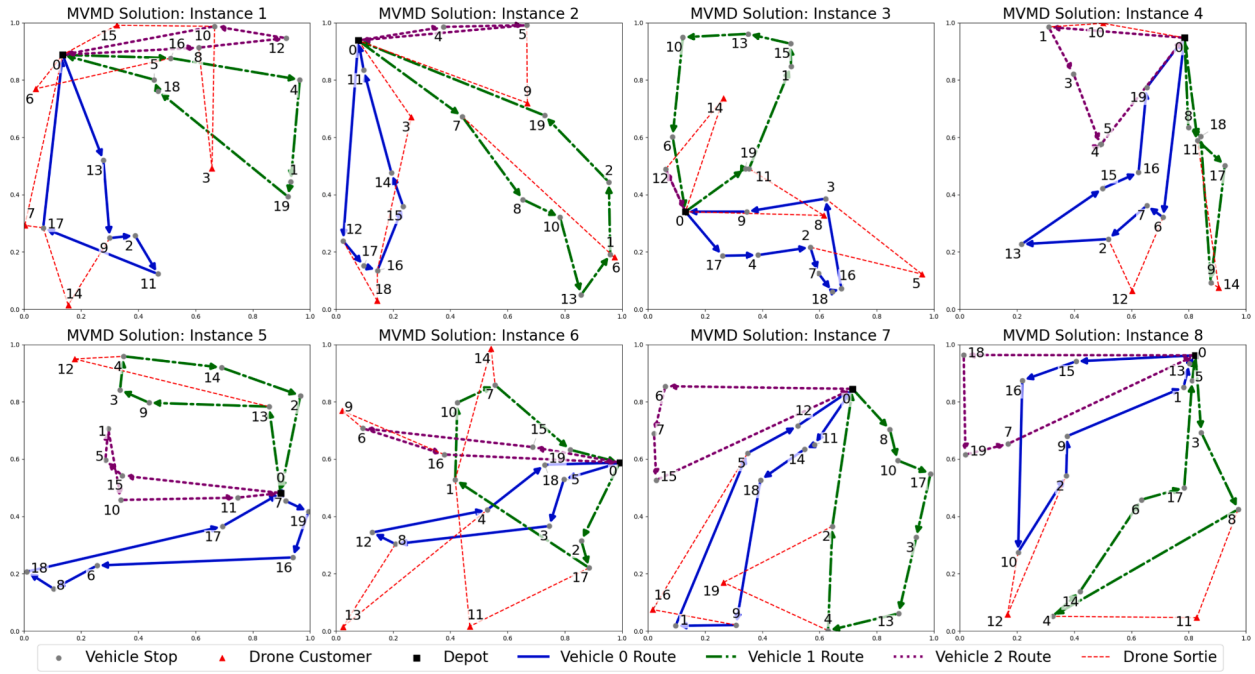


Fig. 7. Solutions obtained by the best-performing heuristic for the MVMD instances.

5.2.2. *Comparison experiments with FSTSP baseline heuristic*

Despite the rapid growth of research on VDCRPs, a unified and publicly available benchmark set for multiple VDCRP variants is still lacking, particularly for problem settings involving tightly coupled vehicle-drone synchronization and joint feasibility constraints. As a result, empirical evaluation in this research area commonly relies on carefully designed synthetic instances and comparisons against representative baseline heuristics. Among existing formulations, the FSTSP introduced by Murray and Chu (2015) is one of the closest canonical problems to the SVSD setting considered in this paper, as it explicitly models the coordination between a vehicle route and drone sorties under endurance constraints. Moreover, the heuristic framework proposed in Murray and Chu (2015) has been widely adopted as a reference baseline in subsequent vehicle-drone routing studies. Motivated by this close structural relationship, we implement an FSTSP baseline heuristic following the original design philosophy and extend it with a payload feasibility check to ensure consistency with the SVSD instance setting. A systematic comparison between this FSTSP baseline and the best-performing SVSD heuristic generated by our LLM-VD framework is then conducted to evaluate relative performance under controlled experimental conditions.

The instances used in this comparison are generated following the same procedure described in Section 5.1. To examine robustness with respect to instance randomness and problem size, we consider three random seeds and three problem sizes (20, 50, and 100 nodes). For clarity of presentation, the instances are organized into three instance-set groups, denoted as ISG1-1 to ISG3-3, where each group corresponds to one random seed and contains three instances of increasing size. The primary performance metric is the makespan, defined as the completion time of the synchronized vehicle–drone delivery plan, and the wall-clock runtime is also reported

Table 4
Comparison of SVSD_SDHR and FSTSP baseline across different problem sizes.

Instances	sizes	makespan			runtime (s)	
		SVSD_SDHR	FSTSP_Baseline	gap*	SVSD_SDHR	FSTSP_Baseline
ISG1-1	20	3.88	4.11	5.44%	0.12	0.10
ISG1-2	50	6.59	6.68	1.36%	3.88	3.23
ISG1-3	100	8.68	9.37	7.31%	82.76	24.36
ISG2-1	20	3.98	4.50	11.61%	0.10	0.11
ISG2-2	50	6.56	6.90	4.92%	5.50	3.40
ISG2-3	100	8.98	9.26	3.03%	107.46	31.10
ISG3-1	20	4.36	4.65	6.10%	0.09	0.09
ISG3-2	50	6.05	6.38	5.10%	4.13	3.47
ISG3-3	100	8.21	8.45	2.90%	93.78	29.30

Notes: The gap column represents the relative objective improvement (in terms of makespan) of SVSD_SDHR compared with the FSTSP baseline, calculated as $(\text{FSTSP_Baseline} - \text{SVSD_SDHR})/\text{FSTSP_Baseline} \times 100\%$.

to reflect computational cost.

The comparative results are summarized in Table 4, and the corresponding illustrative figures are provided in Appendix F. Column SVSD_SDHR denotes the the best-performing SVSD heuristic generated by our LLM-VD framework. Across all nine instances, SVSD_SDHR consistently achieves a smaller makespan than the FSTSP baseline, demonstrating that the advantage of the LLM-generated heuristic is robust to variations in instance randomness and problem size. The relative improvement ranges from 2.90% to 11.61%, indicating that SVSD_SDHR is able to exploit coordination opportunities between vehicles and drones that are not fully captured by the classical baseline. For small- and medium-scale instances with 20 and 50 nodes, the makespan reductions are generally more pronounced, frequently exceeding 5% and reaching double-digit improvements in some cases. This observation suggests that when routing flexibility is relatively high, more refined decisions regarding drone sortie construction and vehicle–drone synchronization can lead to substantial reductions in overall completion time.

As the problem size increases to 100 nodes, the makespan gap between the two methods becomes smaller but remains consistently positive across all random seeds. This trend is expected because larger instances provide more opportunities for both methods to deploy drone sorties, thereby partially mitigating local inefficiencies and reducing the relative impact of individual coordination decisions. Nevertheless, SVSD_SDHR still achieves consistent makespan improvements of approximately 3 to 7% at this scale, indicating that the learned heuristic retains its effectiveness under increased routing complexity and denser synchronization constraints.

In terms of computational efficiency, Table 4 reveals a clear trade-off between solution quality and runtime. The FSTSP baseline exhibits relatively low computational cost across all problem sizes, while SVSD_SDHR requires substantially more runtime, particularly for large-scale instances. This difference is mainly attributable to the richer decision logic embedded in SVSD_SDHR, which involves more elaborate candidate generation, feasibility screening with respect to drone endurance and payload, and a more detailed evaluation of vehicle–drone synchronization effects. Importantly, despite the higher computational effort, the makespan superiority of SVSD_SDHR is observed consistently across all instance-set groups, indicating that the performance gains are systematic rather than driven by isolated instances.

Overall, the comparison with the FSTSP baseline heuristic provides a meaningful and representative reference point in the absence of a shared VDCRPs benchmark. The results in Table 4 demonstrate that the proposed SVSD_SDHR heuristic achieves robust and scalable improvements in makespan across different random seeds and problem sizes, highlighting the effectiveness of LLM-based automatic heuristic design for complex vehicle–drone collaborative routing problems.

5.2.3. *Comparison experiments based on different LLMs*

As shown in Fig. 8, we evaluate the performance of the LLM-VD framework under four different LLMs across the three VDCRP variants. For all backbone models and all problem classes, the score curves decrease monotonically as the number of generations increases, indicating that the evolutionary search consistently improves heuristic quality through iterative refinement. This common downward trend suggests that the proposed evolutionary operators can drive performance gains in a model-agnostic manner. Nevertheless, the four LLMs exhibit clear differences in convergence speed, stability across generations, and the ability to sustain

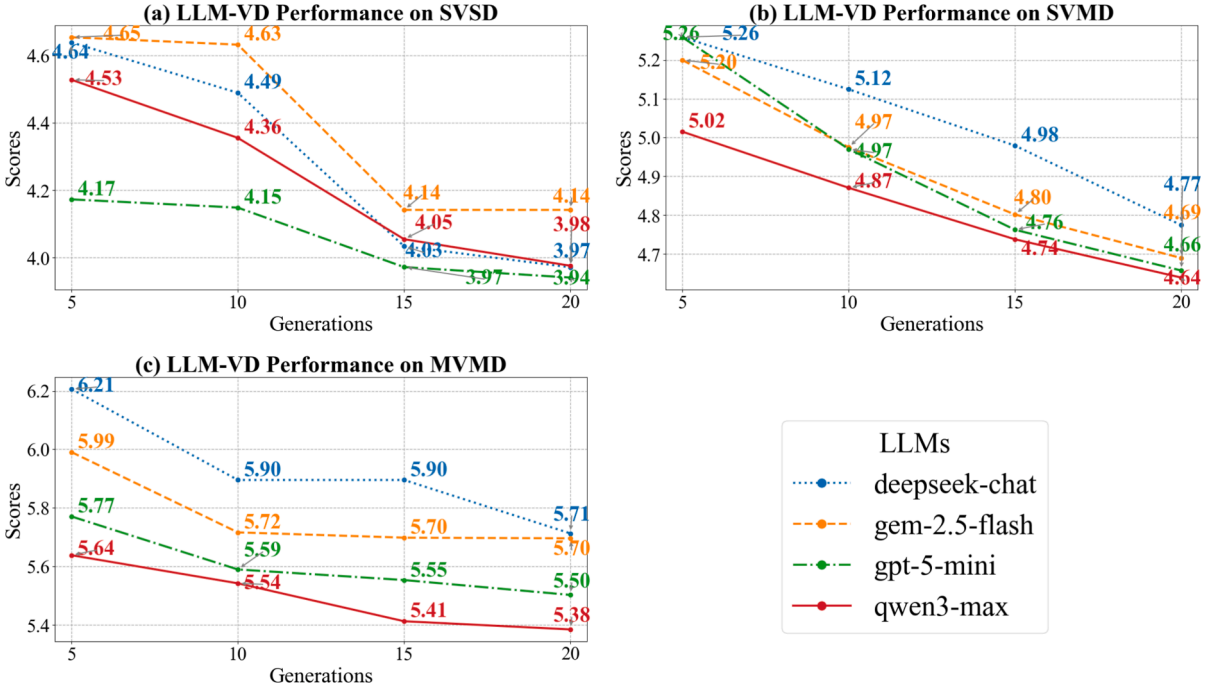


Fig. 8. LLM-VD performance based on different LLMs.

improvements as problem complexity increases.

Overall, *qwen3*-max demonstrates the strongest and most stable optimization capability among the tested models. On the SVSD task, it achieves a near-best final result (3.97), and its advantage becomes more evident on the more complex settings. In particular, *qwen3*-max attains the best final scores on both SVMD (4.64) and MVMD (5.38). This pattern indicates that *qwen3*-max is more effective at supporting the evolutionary search in capturing and consolidating fine-grained structural improvements, which becomes increasingly important when the solution quality is dominated by more intricate vehicle-drone coordination decisions. In contrast, *gpt-5-mini* also performs strongly throughout the evolutionary process, yielding the best result on SVSD (3.84) and consistently competitive convergence trajectories. On SVMD (4.66) and MVMD (5.50), *gpt-5-mini* ranks as the second-best model, suggesting that it remains highly capable of generating effective heuristic refinements, albeit with slightly weaker late-stage gains than *qwen3*-max under higher complexity.

The remaining two models show a similar qualitative improvement trend but exhibit weaker final convergence. *deepseek-chat* maintains stable progress across generations, yet its final results are less competitive than those achieved by *qwen3*-max and *gpt-5-mini*, implying comparatively limited effectiveness in extracting high-impact structural refinements in later evolution stages. *gemini-2.5-flash* exhibits a distinct behavior characterized by relatively limited improvement in the later generations, which results in a larger gap to the best-performing models at convergence. Taken together, these results suggest that while LLM-VD can steadily improve heuristics with any backbone LLM, the achievable solution quality and the persistence of late-stage improvements depend strongly on the model's capability to reason about and refine structured algorithmic components.

In summary, Fig. 8 demonstrates that LLM-VD yields consistent evolutionary improvements across different LLM backbones, but the comparative strengths differ by problem complexity. *qwen3*-max emerges as the most effective backbone overall, particularly on SVMD and MVMD where fine-grained coordination structures matter most. *gpt-5-mini* provides a competitive and robust alternative, achieving the best result on SVSD and remaining close to the best performer on the more complex variants. The observed gaps between models further highlight that stronger reasoning and structural generalization capabilities can translate into more effective evolutionary refinement, especially in high-complexity vehicle-drone routing settings.

5.2.4. *Benefit of self-debugging mechanism*

To assess the effectiveness of the proposed self-debugging mechanisms in LLM-VD, we conducted an ablation study on three VDCRP variants with increasing structural complexity: SVSD, SVMD, and MVMD. The study compares three configurations: the baseline without self-debugging (EoH proposed by Liu et al. (2023a)), self-debugging with failure feedback (SDFF, corresponding to $E = 1$ in Algorithm 1), and self-debugging with heuristic repair (SDHR, $E = 2$ in Algorithm 1).

The results are reported from two complementary perspectives. At the operator level, Table 5 decomposes how feasible heuristics are obtained under SDHR. It distinguishes heuristics generated directly by the evolution operators **E1, E2, M1,** and **M2** from those recovered through the repair operator **R1**. The exact definitions of the E/M, R1, and Total columns follow the notation given in the note of Table 5. At the framework level, Fig. 9 reports the generation-wise success rates of newly produced heuristics under EoH, SDFF, and SDHR. A heuristic is counted as successful only if it passes the full evaluation procedure on all test instances.

Table 5 shows that as problem complexity increases, generating feasible heuristics directly through the evolutionary operators

Table 5
Success rates of new heuristics under SDHR.

Gen.	SVSD by DeepSeek			SVMD by Qwen			MVMD by Gemini		
	E/M (%)	R1	Total (%)	E/M (%)	R1	Total (%)	E/M (%)	R1	Total (%)
2	75	–	75	75	–	75	100	–	100
3	50	1	60	50	1	60	25	–	25
4	75	0	60	50	0	40	100	1	100
5	100	1	100	100	1	100	25	–	25
6	100	–	100	75	–	75	50	1	60
7	100	–	100	100	1	100	75	1	80
8	75	–	75	75	–	75	100	0	80
9	50	1	60	25	0	20	50	–	50
10	100	1	100	75	1	80	25	0	20
11	75	–	75	50	1	60	75	1	80
12	25	1	40	100	1	100	25	1	40
13	100	1	100	100	–	100	75	0	60
14	100	–	100	100	–	100	50	1	60
15	100	–	100	50	–	50	75	0	60
16	75	–	75	100	1	100	50	1	60
17	50	1	60	50	–	50	50	0	40
18	100	1	100	100	1	100	75	1	80
19	75	–	75	75	–	75	25	1	40
Avg.	79.17	0.89	80.83	75.00	0.80	75.56	58.33	0.64	58.89

Note: Column E/M represents the success rate of new heuristics generated by the four evolutionary operators—E1, E2, M1, and M2—excluding the repair operator R1. Column R1 indicates whether the repair operator was used and whether the repaired heuristic passed the evaluation ("-" means not used, "1" means used and successfully repaired, "0" means used but the repair failed). Column Total represents the overall success rate of all new generated heuristics.

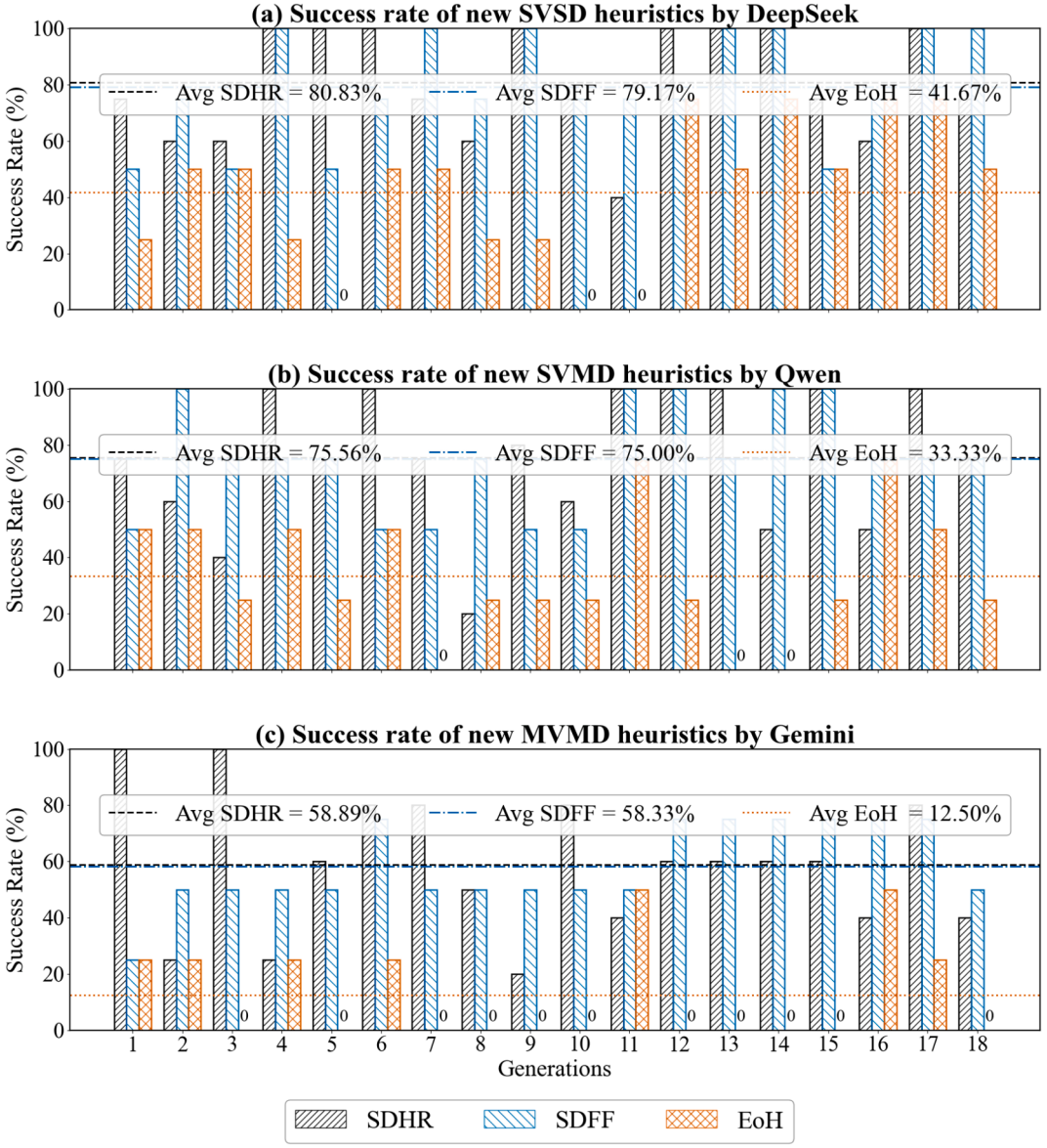


Fig. 9. Success rates of new heuristics under different self-debugging mechanisms.

becomes more difficult. The average success rate of the E/M operators decreases from 79.17% on SVSD to 75.00% on SVMD and further to 58.33% on MVMD. This trend indicates that the additional structural constraints in MVMD more frequently lead to heuristics that are partially valid but infeasible. As a result, more repair attempts are triggered. Although the repair success rate declines from 89.00% on SVSD to 80.00% on SVMD and 64% on MVMD, R1 still recovers a substantial number of failed heuristics. Consequently, SDHR achieves overall success rates of 80.83% on SVSD, 75.56% on SVMD, and 58.89% on MVMD.

The framework-level comparison in Fig. 9 further highlights the importance of self-debugging. Across all three tasks, both SDFF and SDHR clearly outperform the baseline EoH, which lacks any mechanism to exploit evaluation feedback. On SVSD, the average success rate increases from 41.67% under EoH to 79.17% under SDFF and 80.83% under SDHR. On SVMD, the success rate rises from 33.33% to 75.00% and 75.56%, respectively. On the most challenging MVMD problem, the success rate improves from 12.50% under EoH to 58.33% under SDFF and 58.89% under SDHR. These results show that feeding evaluation feedback back into the generation process substantially improves feasibility, especially for high-complexity problems.

A comparison between SDFF and SDHR reveals a more nuanced conclusion. The SDFF mechanism, by feeding back failure causes to the LLM, enables the model to avoid repeating known errors and thereby significantly enhances the quality of directly generated heuristics. However, SDFF cannot address partially valid heuristics that contain localized structural defects. SDHR, through the incorporation of the repair operator R1, introduces the complementary capability of actively transforming failed heuristics into valid

ones. This repair pathway allows SDHR to maintain consistently higher success rates than SDFF across all tasks, especially in high-complexity settings where direct generation becomes increasingly unreliable. The ability to salvage failed heuristics not only improves the overall success rate but also prevents the evolutionary process from prematurely discarding valuable structural information contained in otherwise flawed heuristics.

Overall, the ablation results demonstrate that self-debugging is essential for reliable heuristic generation in LLM-VD. SDFF provides an efficient and low-overhead improvement by exploiting failure feedback, while SDHR further enhances robustness by introducing bounded, targeted heuristic repair. The nearly five-fold improvement over EoH observed on the MVMD problem underscores the critical role of self-debugging, particularly the repair prompt **R1**, in enabling LLMs to reliably produce high-quality heuristics for complex combinatorial optimization tasks.

5.2.5. *Extension to a real-world case with drone recharging constraint*

In this section, we validate the applicability and flexibility of the proposed automatic heuristic design framework using a real-world case study based on the SVSD problem. The instance data is derived from Antwork, a Chinese technology company that developed urban aerial drones in Hangzhou and received the first urban drone operation license from the Civil Aviation Administration of China in 2019. The operational parameters for this case study are based on Antwork's actual operational data. Specifically, the average speed of vehicles is set to 30 km per hour (km/h), the drone speed is set to 45 km/h, the drone's endurance time is set to 0.44 h, and the number of customers is set to 120.

For the extension experiment, we introduce a crucial constraint in practical drone operations: the requirement for drones to recharge between tasks while being synchronized with the vehicle. As outlined in constraint (5) of the SVSD model in Section 3.1, we incorporate a recharging time constraint during the vehicle-drone synchronization period. Based on operational data, the minimum recharging time is set to 0.3 h. Compared to traditional expert-driven heuristic design processes, the proposed LLM-VD framework demonstrates remarkable flexibility and efficiency in handling such dynamic constraints or objectives. When new constraints or decision objectives are introduced, the LLM-VD framework requires only two modifications: (1) updating the task description in natural language as shown in Section 3.2, and (2) modifying the heuristic evaluator in Section 4.2.1 to verify whether the LLM-generated heuristic satisfies the newly added constraints. In the case of the drone recharging constraint, for example, the task description can be extended with the following content in the "Problem Constraints" section: "*After each drone task, check whether the remaining energy is sufficient for the next task. If the remaining energy is insufficient, the drone requires a predefined recharging time before processing the next task.*" Additionally, the heuristic evaluator in Section 4.2.1 is modified to include a check and feedback process for the recharging interval, which is specified as: "*Drone recharging violation: The recharging time between two sorties is less than the predefined recharging time.*" The best-performing heuristic generated under these modifications is named SVSD with recharging constraint (SVSD-RC). Fig. 10 presents the results for both the best-performing SVSD heuristic in Section 5.2.1 and the SVSD-RC heuristic applied to the real-world case. Table 6 shows the drone tasks, task durations and recharging intervals under two heuristics, highlighting the impact of the recharging constraint on task scheduling and performance. It is worth noting that a charging interval of 0 indicates that the drone is performing multiple consecutive tasks within its endurance limit without being recharged.

Compared with the original SVSD heuristic, the SVSD-RC heuristic yields a noticeably smaller number of feasible drone tasks on the

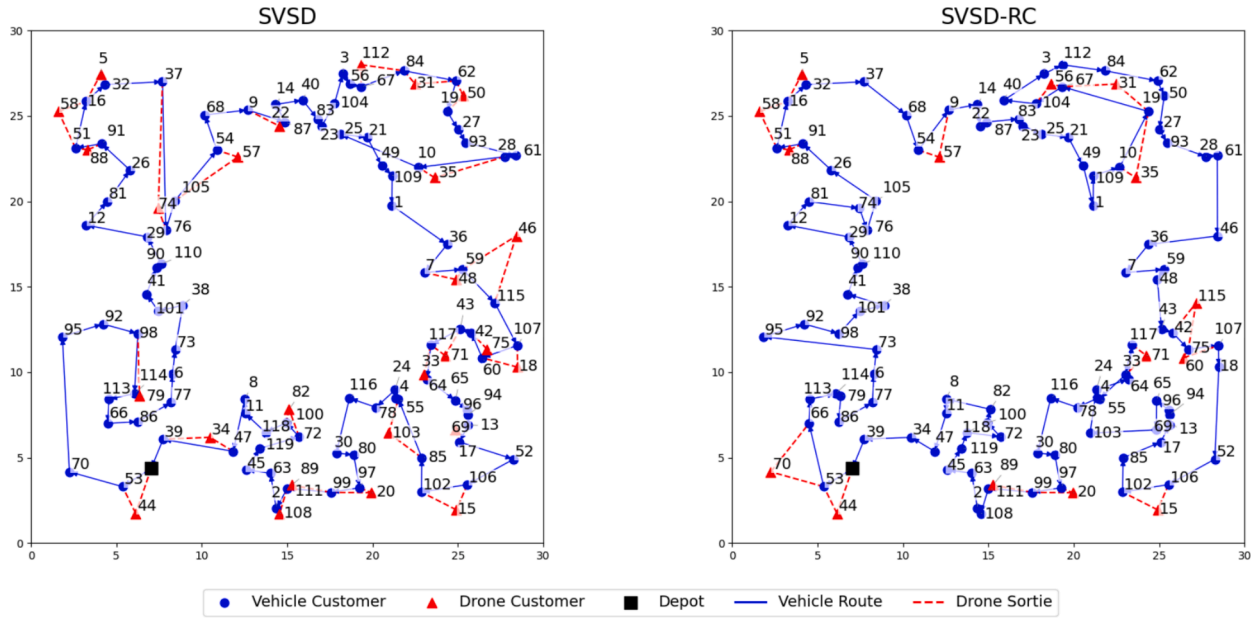


Fig. 10. Solutions obtained by the SVSD and SVSD-RC heuristics on the real-world case.

Table 6
Drone task durations and recharging intervals under SVSD and SVSD-RC heuristics.

SVSD (makespan = 8.15 h, runtime = 219.08 s)			SVSD-RC (makespan = 8.47 h, runtime = 211.02 s)		
Task	Duration (hours)	Interval (hours)	Task	Duration (hours)	Interval (hours)
(0, 44, 53)	0.10	0.33	(0, 44, 53)	0.10	0.00
(98, 79, 114)	0.09	0.55	(53, 70, 66)	0.15	0.56
(91, 88, 51)	0.04	0.00	(91, 88, 51)	0.04	0.00
(51, 58, 16)	0.09	0.00	(51, 58, 16)	0.09	0.00
(16, 5, 32)	0.05	**0.12**	(16, 5, 32)	0.05	0.35
(37, 74, 76)	0.19	**0.08**	(54, 57, 9)	0.09	0.45
(105, 57, 54)	0.13	**0.14**	(10, 35, 19)	0.11	0.00
(9, 22, 87)	0.06	**0.22**	(19, 31, 67)	0.13	0.00
(67, 112, 84)	0.08	0.00	(67, 56, 104)	0.05	0.71
(84, 31, 62)	0.08	0.00	(42, 115, 75)	0.11	0.00
(62, 50, 19)	0.05	**0.20**	(75, 60, 107)	0.06	0.37
(28, 35, 10)	0.12	**0.22**	(106, 15, 102)	0.09	0.31
(7, 48, 59)	0.06	0.00	(117, 71, 33)	0.06	0.35
(59, 46, 115)	0.17	**0.13**	(97, 20, 99)	0.07	0.00
(107, 18, 60)	0.08	0.00	(99, 89, 111)	0.06	—
(60, 75, 42)	0.04	**0.03**	—	—	—
(43, 71, 117)	0.06	0.00	—	—	—
(117, 33, 64)	0.05	**0.17**	—	—	—
(13, 69, 17)	0.03	**0.10**	—	—	—
(106, 15, 102)	0.09	**0.07**	—	—	—
(85, 103, 55)	0.10	**0.25**	—	—	—
(97, 20, 99)	0.07	0.00	—	—	—
(99, 89, 111)	0.06	0.00	—	—	—
(111, 108, 2)	0.04	**0.19**	—	—	—
(72, 82, 100)	0.06	**0.16**	—	—	—
(47, 34, 39)	0.05	—	—	—	—

Note: 1. The Interval column represents the drone recharging time reserved between the task in the corresponding row and the task in the next row; 2. A recharging interval of 0 indicates that the drone is performing multiple continuous tasks within its endurance limit without being recharged.

real-world case, as shown in Fig. 10 and Table 6. This reduction mainly results from the introduction of the drone recharging constraint in SVSD-RC, which requires a minimum recharging interval of 0.3 h between non-consecutive drone tasks. Under SVSD, drone tasks can be scheduled densely along the vehicle route. Table 6 indicates that many non-consecutive tasks have inter-task intervals below 0.3 h (highlighted in bold in the interval column), implicitly assuming immediate redeployment of the drone without accounting for recharging or battery replacement. In contrast, SVSD-RC explicitly distinguishes between consecutive and non-consecutive tasks and enforces the minimum recharging interval for the latter. Task combinations violating this requirement are removed, which directly reduces the number of feasible tasks.

This extension experiment demonstrates that the LLM-VD framework, through minimal adjustments to the task description and evaluator, can efficiently handle real-world constraints like drone recharging time. This confirms the flexibility and adaptability of the proposed framework for solving complex, real-world optimization problems.

6. Conclusions

In this study, we propose LLM-VD, an LLM-driven framework for the automatic heuristic design of VDCRPs. LLM-VD leverages structured prompt strategies to translate complex VDCRP instances into LLM-interpretable natural language representations, enabling automatic generation and evolutionary refinement of heuristics. The key innovation of this study lies in the introduction of two self-debugging mechanisms within the LLM-VD framework. These mechanisms collect feedback from failed heuristic evaluations and iteratively guide the LLM to improve subsequent heuristic generation. The first mechanism integrates failure feedback during the evolutionary process to prevent recurring errors, while the second uses repair prompts to perform code-level corrections on near-feasible heuristics. These self-debugging mechanisms significantly enhance the success rate of heuristic generation, improving the framework's robustness and adaptability. Experimental results demonstrate that LLM-VD produces high-quality heuristics across SVSD, SVMD, and MVMD variants and generalizes well across multiple LLMs. Comparative studies show that self-debugging significantly increases success rates, for example from 41.7% to 81.9% for SVSD, from 33.3% to 75.0% for SVMD, and from 12.5% to 59.7% for MVMD, highlighting its critical role in enhancing heuristic effectiveness and robustness.

The findings provide several theoretical insights. First, a carefully designed structured prompt can significantly enhance the LLM's understanding of different VDCRP variants, leading to more effective heuristics. Second, incorporating failure feedback through the self-debugging mechanism enables the LLM to self-improve, increasing both the effectiveness and robustness of generated heuristics. Third, the LLM-VD framework demonstrates adaptability across different problem variants and LLM models, as well as extensibility to real-world constraints, providing a generalizable approach for automatic heuristic design.

Future research may explore the following directions. First, further investigation into the applicability boundaries and enabling conditions of the LLM-assisted algorithm design methodology, including determining when structured evaluation is sufficient and

when more specialized domain-specific feasibility checkers are needed. Second, an exploration of the broader methodological implications of combining evolutionary computation with LLMs, focusing on addressing potential bottlenecks in sample efficiency and convergence. Lastly, the development of low-cost, high-efficiency heuristic generation frameworks capable of handling dynamic and uncertain factors in more complex real-world scenarios, with the goal of enhancing the practical effectiveness of the framework in operational environments.

CRediT authorship contribution statement

Haiyang Shi: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Lu Zhen:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was supported by the National Natural Science Foundation of China (Grant numbers 72394360, 72394362, and 72361137001), the Project of Science and Technology Commission of Shanghai Municipality, China (23JC1402200), and the China Postdoctoral Science Foundation (2025M780758).

Appendix A. Formulations of SVMD and MVMD

It is worth noting that the SVSD, SVMD, and MVMD problems share a high degree of similarity, particularly in their constraint structures, which often differ only in parameterization. To streamline the presentation, the assumptions and constraints in the SVMD and MVMD formulations that are analogous to those already discussed in the simpler SVSD model are not restated or explained in detail.

A.1. MVMD

Compared with SVSD, SVMD also employs a single vehicle to serve a fixed set of customers, but the vehicle carries multiple drones for delivery. Therefore, in the SVMD problem, it is essential to differentiate the tasks for each drone. The new assumptions of SVMD model is described as following:

- Each customer node must be visited exactly once, either by one drone or the vehicle.

The new notations of SVMD are given below:
Indices and sets
D set of drones $\{1, 2, \cdots, d\}$;
Parameters
t_d^i drone d return time at node i;
Decision variables
y_{id} binary; equal to 1 if the drone d visits node i, otherwise 0.

$$[\mathbf{SVMD}]\text{Min}\left\{\text{Max}\left\{t_v^{c+1}, t_d^{c+1}\right\}\right\} \tag{A1}$$

subject to additional constraints

$$\sum_{\substack{i\in N_v \\ i\neq j}} x_{ij} + \sum_{\substack{d\in D \\ j\in N_d}} y_{jd} = 1 \forall j \in C \tag{A2}$$

$$t_v \geq \sum_{i\in N_v} \sum_{\substack{j\in N_v \\ i\neq j}} \tau_{ij} x_{ij} \tag{A3}$$

$$t_d \geq \sum \left(\tau'_{0,i} + \tau'_{i,c+1}\right) y_{id} \forall d \in D \tag{A4}$$

The objective function (A1) is to minimize the makespan of the entire delivery operation, defined as the time the last device returns to the depot. This is formulated as minimizing the maximum of two values: the final return time of the vehicle t_v^{c+1} and the latest return time t_d^{c+1} among all drones. This objective is subject to several key constraints. Constraint (A2) enforces the customer coverage requirement, ensuring that each node is visited exactly once by either the vehicle or a drone. Concurrently, Constraints (A3) and (A4) define the variables used in the objective by calculating the total operational time required for the vehicle and each drone, respectively, to complete all assigned tasks and return to the depot.

A.2. MVMD

In the MVMD problem, the need for multi-vehicle task allocation necessitates the incorporation of additional constraints related to vehicle assignment. Unlike the other two problems, the MVMD formulation introduces an additional vehicle routing component. Accordingly, the scenario setup is modified as follows:

- Vehicle routes must not overlap or conflict.
- Each customer must be visited exactly once by one drone or one vehicle.

The new notations of MVMD are given below:
Indices and sets.

V set of vehicles $\{1, 2, \cdots, v\}$;
A set of directed arcs $\{(i,j)|i,j \in N, i \neq j\}$;

Decision variables.

x_{ijk} binary; equal to 1 if the vehicle k traverses the arc (i,j) alone, otherwise 0;
y_{ijd} binary; equal to 1 if the drone d traverses the arc (i,j), otherwise 0;
u_{ijk} binary; equal to 1 if the vehicle k traversing the arc (i,j) with drones, otherwise 0.

$$[\mathbf{MVMD}]\text{Min}\{f_1 + f_2\} \tag{A5}$$

$$f_1 = \sum_{(i,j)\in A}\sum_{k\in V} \tau_{ij}\left(x_{ijk} + u_{ijk}\right) \tag{A6}$$

$$f_2 = \sum_{(i,j)\in A}\sum_{d\in D} \tau'_{ij} y_{ijd} \tag{A7}$$

subject to additional constraints

$$\sum_{(i,j)\in A}\sum_{k\in V} x_{ijk} + \sum_{(i,j)\in A}\sum_{d\in D} y_{ijd} + \sum_{(i,j)\in A}\sum_{k\in V} u_{ijk} = 1 \forall j \in C \tag{A8}$$

$$\sum_{(i,j)\in A}\sum_{k\in V} x_{ijk} + \sum_{(i,j)\in A}\sum_{d\in D} y_{ijd} + \sum_{(i,j)\in A}\sum_{k\in V} u_{ijk} = 1 \forall i \in C \tag{A9}$$

$$\sum_{(0,j)\in A}\sum_{k\in V} x_{0jk} + \sum_{(0,j)\in A}\sum_{k\in V} u_{0jk} = \sum_{(j,0)\in A}\sum_{k\in V} x_{j0k} + \sum_{(j,0)\in A}\sum_{k\in V} u_{j0k} \tag{A10}$$

While the objective for the MVMD problem can be to minimize either the total transportation cost or the makespan (total completion time), this study adopts the latter to ensure consistency with the preceding formulations. The objective, as expressed in function (A5), is therefore to minimize the overall delivery time. This objective function comprises two components shown in (A6)-(A7): f_1, which represents the total travel time of the vehicles, and f_2, which denotes the cumulative flight time of all drone sorties. The constraints governing this problem are largely consistent with those of the preceding two models. Specifically, constraints (A8) and (A9) ensure that each customer node is visited exactly once, while constraint (A10) requires that every vehicle both departs from and ultimately returns to the warehouse.

It should be noted that the three types of problem models described above are based on foundational models that typically ignore certain factors such as cargo loading and unloading time or charging time, and therefore do not fully reflect real-world conditions. Different problem variants require distinct modeling approaches to capture their unique operational characteristics. For instance, a key distinction is how fixed durations, such as drone launch or service times, are handled; many models incorporate these as predefined time constants within their constraints. The objectives for these models also vary. The primary goal is typically to minimize the makespan (i.e., total completion time), although some studies focus instead on reducing total operational costs within the objective function.

Appendix B. Template program prompt for SVSD

{template_program for SVSD}

```
template_program = '''
import numpy as np
import random
"Note: Only numpy and random is pre-imported. Other python built-in libraries can be imported inside the function.
**Don't use third-party python libraries**"

def solve_svmd(instance: dict, distance_matrices: dict) -> dict:
    """
    Solve the Single-Vehicle Multiple-Drone (SVMD) routing problem using a heuristic framework.
    Args:
        instance (dict): A dictionary containing the problem instance information.
            'coords': An N x 2 array of coordinates (index 0 is depot, 1..N-1 are customers).
            'demands': An array of demands for N customers (depot demand is 0).
            'num_drones': Number of drones available on vehicle.
            'drone_capacity': Capacity limit per drone.
            'drone_endurance': Maximum flight distance per drone sortie.
        distance_matrices (dict): Distance matrices for vehicle and drone.
            'vehicle': N x N matrix with vehicle travel times (Manhattan).
            'drone': N x N matrix with drone flight times (Euclidean).
    Returns:
        dict:
            'vehicle_route': list of node IDs for vehicle route starting/ending at depot (0).
            'drone_sorties': list of tuples (drone_id, launch_node, customer_node, rendezvous_node).
    """

    # Return an empty solution as placeholder; LLM can optimize on this framework.
    return {'vehicle_route': [], 'drone_sorties': []}
'''
```

Appendix C. The best-performing heuristic for SVSD

```
import math
import random

def solve_svsd(instance: dict, time_matrices: dict) -> dict:
    coords = instance['coords']
    demands = instance['demands']
    drone_capacity = instance['drone_capacity']
    drone_endurance = instance['drone_endurance']
    vehicle_time = time_matrices['vehicle']
    drone_time = time_matrices['drone']
    N = len(coords)
    if N <= 1:
        return {'vehicle_route': [0], 'drone_sorties': []}
    # Helpers
    def route_cost(r):
        c = 0.0
        for a, b in zip(r[:-1], r[1:]):
            c += vehicle_time[a][b]
        return c
    def two_opt_improve(r):
        improved = True
        while improved:
            improved = False
            current_cost = route_cost(r)
            L = len(r)
            for i in range(1, L - 2):
                for k in range(i + 1, L - 1):
                    new_r = r[:i] + list(reversed(r[i:k + 1])) + r[k + 1:]
                    new_cost = route_cost(new_r)
                    if new_cost < current_cost - 1e-9:
                        r = new_r
                        improved = True
                        break
                if improved:
                    break
        return r
```

```
    # Diversified seed builders
    def build_seed_route(seed: int) -> list:
        unvisited = set(range(1, N))
        route = [0]
        current = 0
        rnd = random.Random(seed)
        mode = seed % 3
        while unvisited:
            if mode == 0:
                # nearest-neighbor biased
                sorted_nodes = sorted(list(unvisited), key=lambda j: vehicle_time[current][j])
                top_k = max(1, min(4, len(sorted_nodes)))
                candidates = sorted_nodes[:top_k]
                next_node = rnd.choice(candidates)
            elif mode == 1:
                # farthest among a small sample
                sorted_nodes = sorted(list(unvisited), key=lambda j: -vehicle_time[current][j])
                top_k = max(1, min(4, len(sorted_nodes)))
                candidates = sorted_nodes[:top_k]
                next_node = rnd.choice(candidates)
            else:
                # random among unvisited
                next_node = rnd.choice(list(unvisited))
            route.append(next_node)
            unvisited.remove(next_node)
            current = next_node
        route.append(0)
        route = two_opt_improve(route)
        return route

    BEAM_WIDTH = 8
    candidates = [build_seed_route(seed) for seed in range(BEAM_WIDTH)]
    best_overall_time = float('inf')
    best_vehicle_route = None
    best_sorties = []
```

```
    # Evaluate each seed route
    for route in candidates:
        L = len(route)
        max_i = L - 2
        if max_i <= 0:
            time_est = route_cost(route)
            if time_est < best_overall_time:
                best_overall_time = time_est
                best_vehicle_route = route
                best_sorties = []
            continue

        # Step 2 & 3: enumerate feasible triples
        feasible = [None] * max_i
        for i in range(0, L - 2):
            launch = route[i]
            customer = route[i + 1]
            rendezvous = route[i + 2]
            if customer == 0:
                continue
            if customer >= len(demands) or demands[customer] > drone_capacity:
                continue
            drone_flight = drone_time[launch][customer] + drone_time[customer][rendezvous]
            if drone_flight > drone_endurance:
                continue
            veh_with = vehicle_time[launch][customer] + vehicle_time[customer][rendezvous]
            veh_without = vehicle_time[launch][rendezvous]
            time_saving = veh_with - max(veh_without, drone_flight)
            if time_saving > 1e-9:
                feasible[i] = {
                    'i': i,
                    'launch': launch,
                    'customer': customer,
                    'rendezvous': rendezvous,
                    'time_saving': time_saving
                }
```

```
        # Step 4: Greedy non-overlapping packing for this route
        if max_i > 0:
            feats = [f for f in feasible if f is not None]
            feats.sort(key=lambda x: x['time_saving'], reverse=True)

            used_edges = set()
            selected = []
            for f in feats:
                i = f['i']
                if i in used_edges or (i + 1) in used_edges:
                    continue
                selected.append(f)
                used_edges.add(i)
                used_edges.add(i + 1)
        else:
            selected = []
        sorties = [(f['launch'], f['customer'], f['rendezvous']) for f in selected]
        # Step 5: Remove drone-served middle customers
        if selected:
            to_remove_indices = {f['i'] + 1 for f in selected}
            new_route = [node for idx, node in enumerate(route) if idx not in to_remove_indices]
            if not new_route:
                new_route = [0, 0]
            if new_route[0] != 0:
                new_route.insert(0, 0)
            if new_route[-1] != 0:
                new_route.append(0)
        else:
            new_route = route
        time_without_sums = route_cost(route)
        time_with_drone_sums = sum(f['time_saving'] for f in selected)
        new_time_est = time_without_sums - time_with_drone_sums
        if new_time_est < best_overall_time:
            best_overall_time = new_time_est
            best_vehicle_route = new_route
            best_sorties = sorties
    if best_vehicle_route is None:
        best_vehicle_route = [0, 0]
        best_sorties = []
    return {'vehicle_route': best_vehicle_route, 'drone_sorties': best_sorties}
```

Appendix D. The best-performing heuristic for SVMD

```
import numpy as np
import random

def solve_svmd(instance: dict, time_matrices: dict) -> dict:
    coords = np.array(instance['coords'])
    demands = np.array(instance['demands'])
    num_drones = instance['num_drones']
    drone_capacity = instance['drone_capacity']
    drone_endurance = instance['drone_endurance']
    vehicle_times = np.array(time_matrices['vehicle'])
    drone_times = np.array(time_matrices['drone'])
    N = len(coords)
    depot = 0
    all_customers = set(range(1, N))

    # Step 1: Identify mandatory vehicle customers
    mandatory_vehicle = set()
    for i in all_customers:
        if demands[i] > drone_capacity or drone_times[depot][i] * 2 >
drone_endurance:
            mandatory_vehicle.add(i)
    # Initialize vehicle nodes with depot and mandatory customers
    vehicle_nodes = [depot] + list(mandatory_vehicle)
    clusters = {v: [] for v in vehicle_nodes}
    # Remaining drone-eligible customers
    drone_eligible = [c for c in all_customers if c not in mandatory_vehicle]
    unassigned = set(drone_eligible)

    # Step 2: k-medoids style initialization with score-based selection
    max_centers = len(mandatory_vehicle) + num_drones
    while len(vehicle_nodes) < max_centers + 1 and unassigned:
        best_candidate = None
        best_score = -float('inf')
        for c in unassigned:
            min_drone_time = min(drone_times[center][c] for center in
vehicle_nodes)
            if min_drone_time * 2 > drone_endurance:
                continue
```

```
# Score: prioritize customers with high drone time savings vs vehicle detour
            vehicle_detour = min(
                vehicle_times[u][c] + vehicle_times[c][v] - vehicle_times[u][v]
                for i, u in enumerate(vehicle_nodes)
                for v in vehicle_nodes[i + 1:]
            ) if len(vehicle_nodes) > 1 else vehicle_times[depot][c] * 2
            drone_savings = min_drone_time
            score = drone_savings - 0.5 * vehicle_detour
            if score > best_score:
                best_score = score
                best_candidate = c
        if best_candidate is None:
            break
        vehicle_nodes.append(best_candidate)
        clusters[best_candidate] = []
        unassigned.remove(best_candidate)
    # Assign remaining customers to best feasible center using score
    for customer in list(unassigned):
        best_center = None
        best_score = -float('inf')
        for center in vehicle_nodes:
            if len(clusters[center]) >= num_drones:
                continue
            rt = drone_times[center][customer] * 2
            if demands[customer] <= drone_capacity and rt <= drone_endurance:
                score = -drone_times[center][customer]  # minimize drone time
                if score > best_score:
                    best_score = score
                    best_center = center
        if best_center is not None:
            clusters[best_center].append(customer)
        else:
            vehicle_nodes.append(customer)
            clusters[customer] = []
    # Ensure depot is first
    if vehicle_nodes[0] != depot:
        vehicle_nodes.remove(depot)
        vehicle_nodes.insert(0, depot)
```

```
# Step 3: Build initial vehicle route using nearest neighbor

    if len(vehicle_nodes) == 1:
        vehicle_route = [depot, depot]
    else:
        unvisited = set(vehicle_nodes) - {depot}
        route = [depot]
        current = depot
        while unvisited:
            next_node = min(unvisited, key=lambda x: vehicle_times[current][x])
            route.append(next_node)
            unvisited.remove(next_node)
            current = next_node
        route.append(depot)
        vehicle_route = route

    # Step 4: Local search refinement – reassign customers after route improvement

    def two_opt(route, dist_matrix):
        best = route[:]
        improved = True
        while improved:
            improved = False
            for i in range(1, len(best) - 2):
                for j in range(i + 1, len(best) - 1):
                    new_route = best[:i] + best[i:j + 1][::-1] + best[j + 1:]
                    old_dist = sum(dist_matrix[best[k]][best[k + 1]] for k in range(len(best) - 1))
                    new_dist = sum(dist_matrix[new_route[k]][new_route[k + 1]] for k in
range(len(new_route) - 1))
                    if new_dist < old_dist:
                        best = new_route
                        improved = True
                        break
                if improved:
                    break
        return best

    if len(vehicle_route) > 3:
        vehicle_route = two_opt(vehicle_route, vehicle_times)
```

```
# Update clusters based on refined route
    refined_vehicle_nodes = [node for node in vehicle_route if node != depot] + [depot]
    refined_clusters = {node: [] for node in refined_vehicle_nodes}
    for center, customers in clusters.items():
        if center in refined_clusters:
            refined_clusters[center].extend(customers)
        else:
            for c in customers:
                best_center = None
                best_score = -float('inf')
                for rc in refined_vehicle_nodes:
                    if len(refined_clusters[rc]) >= num_drones:
                        continue
                    rt = drone_times[rc][c] * 2
                    if demands[c] <= drone_capacity and rt <= drone_endurance:
                        score = -drone_times[rc][c]
                        if score > best_score:
                            best_score = score
                            best_center = rc
                if best_center is not None:
                    refined_clusters[best_center].append(c)
                else:
                    if c not in refined_vehicle_nodes:
                        refined_vehicle_nodes.append(c)
                        refined_clusters[c] = []
    # Final vehicle route
    final_vehicle_nodes = [depot] + [n for n in vehicle_route[1:-1] if n in refined_clusters] + [depot]
    vehicle_route = final_vehicle_nodes
    # Generate drone sorties
    drone_sorties = []
    for center in vehicle_route[:-1]:
        customers = refined_clusters.get(center, [])
        for idx, customer in enumerate(customers):
            drone_id = idx % num_drones
            drone_sorties.append((drone_id, center, customer, center))
    return {
        'vehicle_route': vehicle_route,
        'drone_sorties': drone_sorties
    }
```

Appendix E. The best-performing heuristic for MVMD

```
import numpy as np
import random
import math

def solve_mvmd(instance: dict, time_matrices: dict) -> dict:
    coords = instance['coords']
    demands = instance['demands']
    num_vehicles = instance['num_vehicles']
    vehicle_capacity = instance['vehicle_capacity']
    drone_capacity = instance['drone_capacity']
    drone_endurance = instance['drone_endurance']
    N = len(coords)
    depot = 0
    customers = list(range(1, N))

    # Step 1: Initial assignment via sweep
    angles = []
    depot_x, depot_y = coords[depot]
    for i in customers:
        x, y = coords[i]
        angle = math.atan2(y - depot_y, x - depot_x)
        angles.append((angle, i))
    angles.sort()

    vehicle_customers = [[] for _ in range(num_vehicles)]
    vehicle_loads = [0] * num_vehicles

    idx = 0
    for _, customer in angles:
        assigned = False
        for offset in range(num_vehicles):
            v = (idx + offset) % num_vehicles
            if vehicle_loads[v] + demands[customer] <= vehicle_capacity:
                vehicle_customers[v].append(customer)
                vehicle_loads[v] += demands[customer]
                idx = v
                assigned = True
                break
```

```
        if not assigned:
            candidates = [(vehicle_loads[v], v) for v in range(num_vehicles) if
                          vehicle_loads[v] + demands[customer] <= vehicle_capacity]
            if candidates:
                _, v = min(candidates)
                vehicle_customers[v].append(customer)
                vehicle_loads[v] += demands[customer]
                idx = v
            else:
                v = idx % num_vehicles
                vehicle_customers[v].append(customer)
                vehicle_loads[v] += demands[customer]
                idx += 1
    vehicle_time = time_matrices['vehicle']
    drone_time = time_matrices['drone']
    def build_route_and_sorties(v_id, cust_list):
        if not cust_list:
            return [0, 0], []
        unvisited = set(cust_list)
        current = depot
        route = [depot]
        while unvisited:
            next_node = min(unvisited, key=lambda x: vehicle_time[current][x])
            route.append(next_node)
            unvisited.remove(next_node)
            current = next_node
        route.append(depot)
        def two_opt(r):
            improved = True
            while improved:
                improved = False
                best_dist = sum(vehicle_time[r[i]][r[i + 1]] for i in range(len(r) - 1))
                for i in range(1, len(r) - 2):
                    for k in range(i + 1, len(r) - 1):
                        new_r = r[:i] + r[i:k + 1][::-1] + r[k + 1:]
                        new_dist = sum(vehicle_time[new_r[j]][new_r[j + 1]] for j in range(len(new_r) - 1))
                        if new_dist < best_dist:
                            r = new_r
                            improved = True
                            break
                    if improved:
                        break
            return r
```

```
        route = two_opt(route)

        n = len(route)
        feasible_sorties = []
        for launch_idx in range(n):
            launch_node = route[launch_idx]
            for rendezvous_idx in range(launch_idx + 2, n):
                rendezvous_node = route[rendezvous_idx]
                if launch_node == depot and rendezvous_node == depot:
                    continue
                for mid_idx in range(launch_idx + 1, rendezvous_idx):
                    customer = route[mid_idx]
                    if demands[customer] > drone_capacity:
                        continue
                    d_time = drone_time[launch_node][customer] + drone_time[customer][rendezvous_node]
                    if d_time > drone_endurance:
                        continue

                    seg_with = sum(vehicle_time[route[i]][route[i + 1]] for i in range(launch_idx, rendezvous_idx))
                    temp_route = route[:mid_idx] + route[mid_idx + 1:]
                    seg_without = sum(
                        vehicle_time[temp_route[i]][temp_route[i + 1]] for i in range(launch_idx, rendezvous_idx - 1))
                    max_time = max(seg_without, d_time)
                    time_saving = seg_with - max_time
                    if time_saving > 0:
                        feasible_sorties.append(
                            (time_saving, launch_idx, rendezvous_idx, launch_node, customer, rendezvous_node))

        feasible_sorties.sort(key=lambda x: -x[0])
        selected_intervals = []
        selected_sorties = []
        drone_customers = set()

        for ts, l_idx, r_idx, ln, cust, rn in feasible_sorties:
            overlap = False
            for (a, b) in selected_intervals:
                if not (r_idx <= a or l_idx >= b):
                    overlap = True
                    break
            if not overlap and cust not in drone_customers:
                selected_intervals.append((l_idx, r_idx))
                selected_sorties.append((v_id, ln, cust, rn))
                drone_customers.add(cust)

        final_route = [node for node in route if node == depot or node not in drone_customers]
        return final_route, selected_sorties
```

```
    # Build initial solution
    vehicle_routes = []
    drone_sorties = []
    for v_id, cust_list in enumerate(vehicle_customers):
        route, sorties = build_route_and_sorties(v_id, cust_list)
        vehicle_routes.append(route)
        drone_sorties.extend(sorties)
    # Local search: try moving one customer from one vehicle to another if capacity allows
    improved = True
    while improved:
        improved = False
        for src_v in range(num_vehicles):
            for dst_v in range(num_vehicles):
                if src_v == dst_v:
                    continue
                for cust in vehicle_customers[src_v]:
                    if vehicle_loads[dst_v] + demands[cust] <= vehicle_capacity and vehicle_loads[src_v] - demands[cust] >= 0:
                        # Try move
                        new_src_cust = [c for c in vehicle_customers[src_v] if c != cust]
                        new_dst_cust = vehicle_customers[dst_v] + [cust]
                        new_src_route, new_src_sorties = build_route_and_sorties(src_v, new_src_cust)
                        new_dst_route, new_dst_sorties = build_route_and_sorties(dst_v, new_dst_cust)
                        old_src_route = vehicle_routes[src_v]
                        old_dst_route = vehicle_routes[dst_v]
                        old_src_sortie_set = {s for s in drone_sorties if s[0] == src_v}
                        old_dst_sortie_set = {s for s in drone_sorties if s[0] == dst_v}

                        # Compute completion times
                        def compute_completion_time(route, sorties_for_v):
                            if len(route) <= 2:
                                return 0.0
                            cust_to_sortie = {}
                            for (_, l, c, r) in sorties_for_v:
                                cust_to_sortie[c] = (l, r)
                            total_time = 0.0
                            for i in range(len(route) - 1):
                                seg_time = vehicle_time[route[i]][route[i + 1]]
                                max_drone_time = 0.0
                                for j in range(i + 1, len(route)):
                                    if route[j] in cust_to_sortie:
                                        l_node, r_node = cust_to_sortie[route[j]]
                                        if route[i] == l_node or route[i + 1] == r_node or (
                                                route[i] != r_node and route[i + 1] != l_node):
                                            pass
                                total_time += seg_time
                            return total_time
```

```
            # Approximate improvement by comparing route lengths (since exact waiting is complex)
                old_len = sum(vehicle_time[old_src_route[i]][old_src_route[i + 1]] for i in
                              range(len(old_src_route) - 1)) + \
                          sum(vehicle_time[old_dst_route[i]][old_dst_route[i + 1]] for i in
                              range(len(old_dst_route) - 1))
                new_len = sum(vehicle_time[new_src_route[i]][new_src_route[i + 1]] for i in
                              range(len(new_src_route) - 1)) + \
                          sum(vehicle_time[new_dst_route[i]][new_dst_route[i + 1]] for i in
                              range(len(new_dst_route) - 1))

                if new_len < old_len:
                    vehicle_customers[src_v] = new_src_cust
                    vehicle_customers[dst_v] = new_dst_cust
                    vehicle_loads[src_v] -= demands[cust]
                    vehicle_loads[dst_v] += demands[cust]
                    vehicle_routes[src_v] = new_src_route
                    vehicle_routes[dst_v] = new_dst_route
                    drone_sorties = [s for s in drone_sorties if s[0] not in (src_v, dst_v)]
                    drone_sorties.extend(new_src_sorties)
                    drone_sorties.extend(new_dst_sorties)
                    improved = True
                    break
            if improved:
                break
        if improved:
            break

    return {
        'vehicle_routes': vehicle_routes,
        'drone_sorties': drone_sorties
    }
```

Appendix F. Solution comparison with FSTSP baseline heuristic

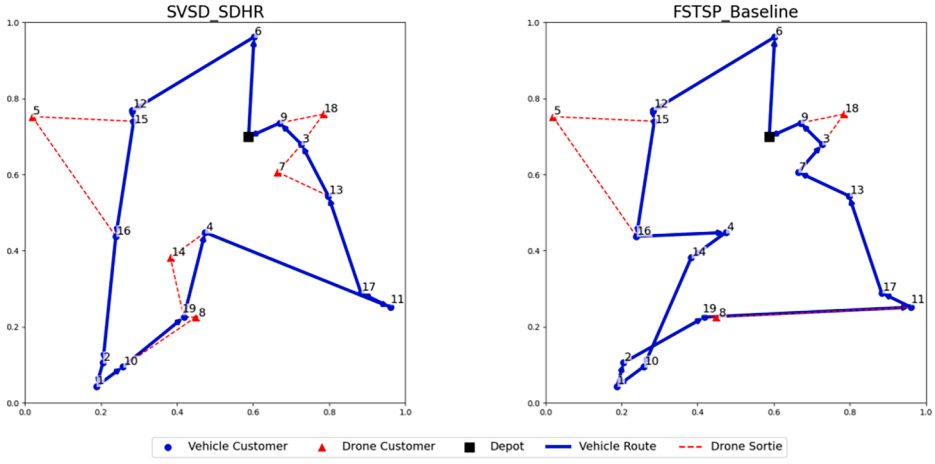


Fig. F1. Solution comparison with FSTSP baseline heuristic on ISG1-1.

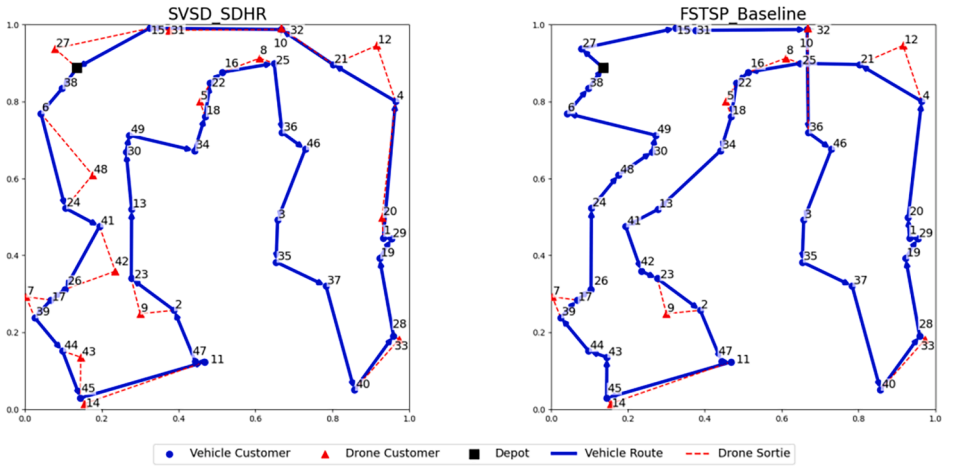


Fig. F2. Solution comparison with FSTSP baseline heuristic on ISG2-2.

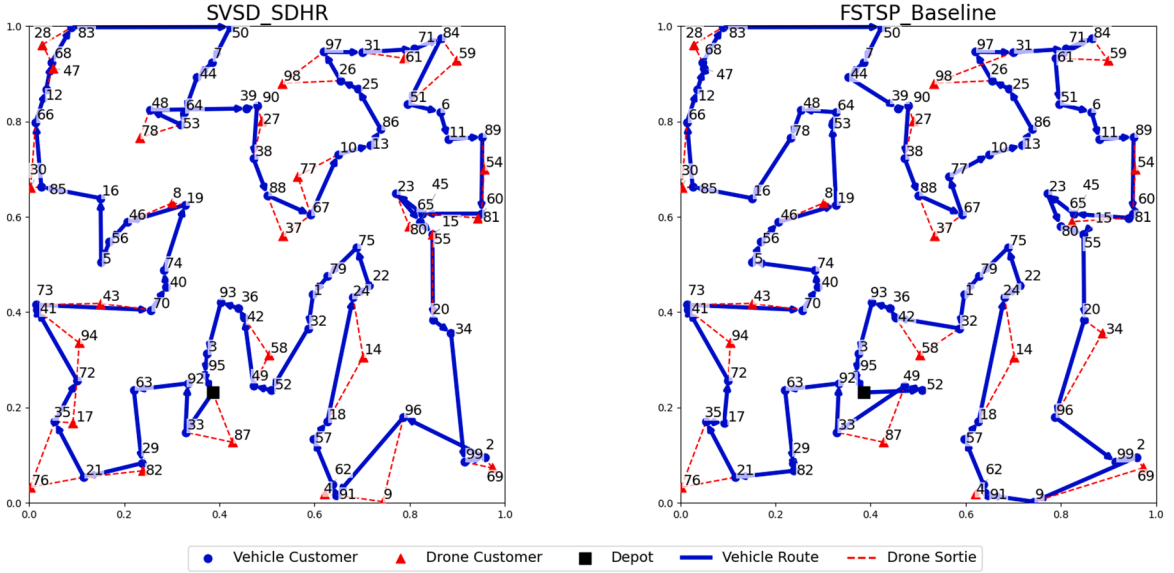


Fig. F3. Solution comparison with FSTSP baseline heuristic on ISG3-3.

Data availability

Data will be made available on request.

References

Agatz, N., Bouman, P., Schmidt, M., 2018. Optimization approaches for the traveling salesman problem with drone. Transp. Sci. 52 (4), 965–981.
Bi, Z., Guo, X., Wang, J., Qin, S., Liu, G., 2023. Deep reinforcement learning for truck-drone delivery problem. Drones 7 (7), 445.
Chang, Y., Lee, H., 2018. Optimal delivery routing with wider drone-delivery areas along a shorter truck-route. Expert Syst. Appl. 104, 307–317.
Chen, C., Demir, E., Huang, Y., 2021. An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and delivery robots. Eur. J. Oper. Res. 294 (3), 1164–1180.
Chung, S., Sah, B., Lee, J., 2020. Optimization for drone and drone-truck combined operations: a review of the state of the art and future directions. Comput. Oper. Res. 123, 105004.
Dang, S., Liu, Y., Luo, Z., Liu, Z., Shi, J., 2024. A survey of the routing problem for cooperated trucks and drones. Drones 8 (10), 550.
De Freitas, J., Penna, P., 2020. A variable neighborhood search for flying sidekick traveling salesman problem. Int. Trans. Oper. Res. 27 (1), 267–290.
Gao, J., Zhen, L., Laporte, G., He, X., 2023a. Scheduling trucks and drones for cooperative deliveries. Transp. Res. Part E Logist. Transp. Rev. 178, 103267.
Gao, J., Zhen, L., Wang, S., 2023b. Multi-trucks-and-drones cooperative pickup and delivery problem. Transp. Res. Part C Emerging Technol. 157, 104407.
Gu, R., Poon, M., Luo, Z., Liu, Y., Liu, Z., 2022. A hierarchical solution evaluation method and a hybrid algorithm for the vehicle routing problem with drones and multiple visits. Transp. Res. Part C Emerging Technol. 141, 103733.

Ha, Q., Deville, Y., Pham, Q., Ha, M., 2018. On the min-cost traveling salesman problem with drone. Transp. Res. Part C Emerging Technol. 86, 597–621.
Huang, Z., Shi, G., Sukhatme, G., 2024. From words to routes: applying large language models to vehicle routing. arXiv, 2403.10795.
Jawahar, G., Abdul-Mageed, M., Lakshmanan, L., Ding, D., 2024. LLM performance predictors are good initializers for architecture search. arXiv, 2310.16712.
Jiang, X., Wu, Y., Wang, Y., Zhang, Y., 2024. Bridging large language models and optimization: a unified framework for text-attributed combinatorial optimization. arXiv, 2408.12214.
Kang, M., Lee, C., 2021. An exact algorithm for heterogeneous drone-truck routing problem. Transp. Sci. 55 (5), 1088–1112.
Kloster, K., Moeini, M., Vigo, D., Wendt, O., 2023. The multiple traveling salesman problem in presence of drone- and robot-supported packet stations. Eur. J. Oper. Res. 305 (2), 630–643.
Lange R, Tian Y, Tang Y (2024) Large language models as evolution strategies. *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2024) Companion*.
Li, K., Liu, F., Wang, Z., Tong, X., Han, X., Yuan, M., Zhang, Q., 2025. ARS: automatic routing solver with large language models. arXiv, 2502.15359.
Li, X., Yan, P., Yu, K., Li, P., Liu, Y., 2024. Parcel consolidation approach and routing algorithm for last-mile delivery by unmanned aerial vehicles. Expert Syst. Appl. 238, 122149.
Liu F, Tong X, Yuan M, Lin X, Luo F, Wang Z, Lu Z, Zhang Q (2023a) Evolution of heuristics: towards efficient automatic algorithm design using large language model. *ICML 2024 (Proceedings) / arXiv:* 2401.02051.
Liu, F., Tong, X., Yuan, M., Zhang, Q., 2023. Algorithm evolution using large language model. arXiv, 2311.15249.
Liu, F., Yao, Y., Guo, P., Yang, Z., Zhao, Z., Lin, X., Yuan, M., Lu, Z., Wang, Z., Zhang, Q., 2024. A systematic survey on large language models for algorithm design. arXiv, 2410.14716.
Lu, J., Liu, Y., Jiang, C., Wu, W., 2025. Truck-drone joint delivery network for rural area: optimization and implications. Transp. Policy 163, 273–284.
Meng, S., Wang, Y., Yang, C., Peng, N., Chang, K., 2024. LLM-A: large language model enhanced incremental heuristic search on path planning. arXiv, 2407.02511.
Montemanni, R., Dell'Amico, M., Corsini, A., 2024. Parallel drone scheduling vehicle routing problems with collective drones. Comput. Oper. Res. 163, 106514.
Moshref-Javadi, M., Lee, S., Winkenbach, M., 2020. Design and evaluation of a multi-trip delivery model with truck and drones. Transp. Res. Part E Logist. Transp. Rev. 136, 101887.
Moshref-Javadi, M., Winkenbach, M., 2021. Applications and research avenues for drone-based models in logistics: a classification and review. Expert Syst. Appl. 177, 114854.
Murray, C., Chu, A., 2015. The flying sidekick traveling salesman problem: optimization of drone-assisted parcel delivery. Transp. Res. Part C Emerging Technol. 54, 86–109.
Paul, A., Levin, M., Waller, S., Rey, D., 2025. Data-driven optimization for drone delivery service planning with online demand. Transp. Res. Part E Logist. Transp. Rev. 198, 104095.
Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M., Dupont, E., Ruiz, F., Ellenberg, J., Wang, P., Fawzi, O., Kohli, P., Fawzi, A., 2024. Mathematical discoveries from program search with large language models. Nature 625 (7995), 468–475.
Rong, R., Ma, S., Ren, N., Lin, Q., Jia, N., 2025. Generative artificial intelligence in intelligent transportation systems: a systematic review of applications. Front. Eng. Manage. 12 (4), 1020–1036.
Sacramento, D., Pisinger, D., Ropke, S., 2019. An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones. Transp. Res. Part C Emerging Technol. 102, 289–315.
Salama, M., Srinivas, S., 2022. Collaborative truck multi-drone routing and scheduling problem: package delivery with flexible launch and recovery sites. Transp. Res. Part E Logist. Transp. Rev. 164, 102788.
Savuran, H., Karakaya, M., 2016. Efficient route planning for an unmanned air vehicle deployed on a moving carrier. Soft. Comput. 20 (7), 2905–2920.
Schmidt, J., Tilk, C., Irnich, S., 2025. Exact solution of the vehicle routing problem with drones. Transp. Sci. 59 (1), 60–80.
Stein, N., Back, T., 2024. LLaMEA: a large language model evolutionary algorithm for automatically generating metaheuristics. arXiv, 2405.20132.
Tamke, F., Buscher, U., 2021. A branch-and-cut algorithm for the vehicle routing problem with drones. Transp. Res. B Methodol. 144, 174–203.
Tiniç, G., Karasan, O., Kara, B., Campbell, J., Ozel, A., 2023. Exact solution approaches for the minimum total cost traveling salesman problem with multiple drones. Transp. Res. B Methodol. 168, 81–123.
Wang, L., Duan, P., He, Z., Lyu, C., Chen, X., Zheng, N., Yao, L., Ma, Z., 2025. Agentic Large Language Models for day-to-day route choices. Transp. Res. Part C Emerging Technol. 180, 105307.
Wang, S., Zhu, Y., Li, Z., Wang, Y., Li, L., He, Z., 2023. ChatGPT as your vehicle co-pilot: an initial attempt. IEEE Trans. Intell. Veh. 8 (12), 4706–4719.
Wang, Y., Wang, Z., Hu, X., Xue, G., Guan, X., 2022. Truck–drone hybrid routing problem with time-dependent road travel time. Transp. Res. Part C Emerging Technol. 144, 103901.
Wang, Z., Sheu, J., 2019. Vehicle routing problem with drones. Transp. Res. B Methodol. 122, 350–364.
Wu, H., Li, G., Ivanov, D., 2025. The transformative power of generative AI for supply chain management: theoretical framework and agenda. Front. Eng. Manage. 12 (2), 425–433.
Wu, Y., Wang, S., Zhen, L., Laporte, G., 2023. Integrating operations research into green logistics: a review. Front. Eng. Manage. 10 (3), 517–533.
Wu, X., 2024. Enhancing Q-learning with large language model heuristics. arXiv, 2405.03341.
Yang C, Wang X, Lu Y, Liu H, Le Q, Zhou D, Chen X (2023) Large language models as optimizers. *ICLR 2024 (Proceedings) / arXiv:* 2309.03409.
Yang, L., Zhen, L., 2025. Exact algorithm for autonomous dump truck routing in open-pit mines considering coal production. Front. Eng. Manage. 12 (4), 1005–1019.
Ye H, Wang J, Cao Z, Berto F, Hua C, Kim H, Park J, Song G (2024) ReEvo: Large language models as hyper-heuristics with reflective evolution. *NeurIPS 2024 (Proceedings) / arXiv:* 2402.01145.
Yin, Y., Yang, Y., Yu, Y., Wang, D., Cheng, T., 2023. Robust vehicle routing with drones under uncertain demands and truck travel times in humanitarian logistics. Transp. Res. B Methodol. 174, 102781.
Zhen, L., Gao, J., Wang, S., Laporte, G., Yue, X., 2025. Optimizing an on-demand delivery mode based on trucks and drones. Transp. Sci. 59 (5), 1008–1031.
Zhou, J., Yi, J., Yang, Z., Pu, H., Li, X., Luo, J., Gao, L., 2025a. A survey on vehicle–drone cooperative delivery operations optimization: models, methods, and future research directions. Swarm Evol. Comput. 92, 101780.
Zhou, Q., Wu, J., Zhu, M., Zhou, Y., Xiao, F., Zhang, Y., 2025b. LLM-QL: a LLM-enhanced Q-learning approach for scheduling multiple parallel drones. IEEE Trans. Knowl. Data Eng. 37 (9), 5393–5406.